SocialSphere

Full-Stack Interview Preparation

React · Node · Express · MongoDB · Redis · Socket.IO

Architecture, language fundamentals, every bug explained,
design decisions and interview Q&A

Generated 2026-11-11

Contents

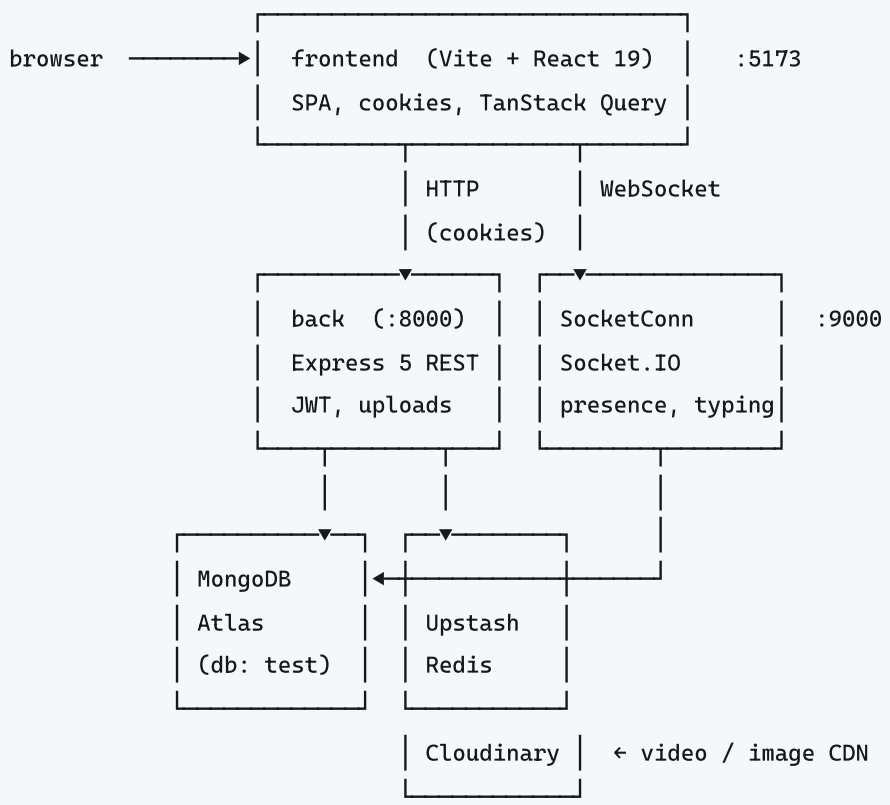
1. 1 — Architecture & Running It
2. 2 — The JavaScript This Project Depends On
3. 3 — Node & Express
4. 4 — MongoDB & Mongoose
5. 5 — React & The Frontend
6. 6 — Authentication
7. 7 — Caching, Rate Limiting, Realtime & Scale
8. 8 — Every Bug, Explained
9. 9 — Design Decisions
10. 10 — Interview Q&A

1 — Architecture & Running It

> Part 1 of the SocialSphere / preytweet interview-prep series.
> `01` architecture · `02` JavaScript · `03` Node & Express · `04` MongoDB
> `05` React · `06` Auth · `07` Caching & scale · `08` Every bug, explained
> `09` Design decisions · `10` Interview Q&A

1.1 What this project actually is

Three deployable units sharing one MongoDB Atlas cluster.



Unit	Folder	Port	Responsibility
Frontend	`frontend/`	5173	React SPA
REST API	`back/`	8000	auth, users, videos, message persistence
Realtime	`SocketConn/`	9000	presence, typing, live delivery

The split to be able to defend in an interview: messages are *written*

through the REST API (so they are validated, rate-limited and durably stored)

and *delivered* through the socket service. The socket layer is a transport, not a source of truth. If the socket service dies, nothing is lost — the next `GET /api/v1/socket/:receiverId` still returns every message. That is the whole reason the two are separate processes.

1.2 Running it locally

Three terminals.

```
cd back && npm run dev
```

```
cd SocketConn && npm run dev
```

```
cd frontend && npm run dev
```

Then open .

Demo account

```
username: demouser
password: demo1234
```

Recreate or reset it any time:

```
cd back && node script/seed-test-user.js
```

That script writes straight to Mongo, so it does not need Cloudinary. It assigns the plaintext password and calls `.save()`, which means the model's `pre("save")` hook hashes it exactly as a real registration would.

Health checks

```
curl http://localhost:8000/health
```

```
curl http://localhost:9000/health
```

The API's `/health` reports whether Mongo is connected and whether Redis is live or has degraded to the in-memory fallback. That distinction matters: the API will happily serve traffic with Redis down, and without this endpoint you would not know you were running uncached.

1.3 Environment configuration

`back/.env`

Key	Purpose
`PORT`	API port (8000)
`CORS_ORIGIN`	comma-separated allow-list
`MONGODB_URL`	Atlas connection string
`DB_NAME`	database name — defaults to `test`
`ACCESS_TOKEN_SECURITY` / `_EXPIRY`	access-JWT signing
`REFRESH_TOKEN_SECURITY` / `_EXPIRY`	refresh-JWT signing
`CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_SECRET_KEY`	media uploads
`REDIS_URL`	Upstash; optional — falls back to memory
`ELASTIC_URL`	optional; falls back to Mongo regex search

`frontend/.env.local` (gitignored, overrides `.env`):

```
VITE_BACKEND=http://localhost:8000
VITE_SOCKET_BACKEND=http://localhost:9000
```

Why `.env.local` instead of editing `.env`: Vite loads `.env` then `.env.local`, and the latter wins. So `.env` can keep pointing at the deployed Render services for `npm run build`, while local development silently overrides it. No more commenting lines in and out.

The `DB_NAME` trap

The original code built the connection string like this:

```
mongoose.connect(`${process.env.MONGODB_URL}/${DBNAME}`)
```

with `DBNAME = "TEST VERSION"`. But `MONGODB_URL` already ends in a query string:

```
mongodb://user:pass@host:27017/?ssl=true&replicaSet=...&appName=Cluster0
```

Appending `/TEST VERSION` lands it **after** the `?`, so it became part of the `appName` parameter. The driver saw an empty path and silently used its default database, `test`.

So `DBNAME` never did anything, and all the real data lives in `test`. The fix passes the name as an option instead — and deliberately keeps `test` as the default so no data is orphaned:

```
mongoose.connect(env.MONGODB_URL, { dbName: env.DB_NAME })  // DB_NAME = "test"
```

Interview point: a connection string is a URI. Concatenating onto a URI that already has a query string is always a bug. Use the library's options object, or `new URL()`.

1.4 Request lifecycle

Walk this end-to-end in an interview; it shows you understand the whole stack.

```
POST /api/v1/videos/createvideo
   │
   ├─► cors()                    origin in allow-list? echo it back
   ├─► express.json()            (skipped — multipart body)
   ├─► cookieParser()            req.cookies.accessToken
   │
   ├─► verifyJWT                 jwt.verify → User.findById → req.user
   ├─► rateLimit                 INCR rate:user:<id>, EXPIRE 20s
   ├─► multer.fields()           stream files → tmp/uploads/<random>
   │
   ├─► createVideo controller
   │      ├ validate title/description/files
   │      ├ uploadOnCloudinary ×2  (then always unlink the temp file)
   │      ├ Video.create
   │      ├ indexVideo             (fire-and-forget → Elasticsearch)
   │      └ invalidate("videos:*") (drop cached feed pages)
   │
   ├─► res.status(201).json(apiResponse)
   │
   └─► (on throw) asyncHandler → next(err) → global error handler
```

Everything after `verifyJWT` can assume `req.user` exists. Everything that throws lands in one error handler. That uniformity is the point of the `asyncHandler` + `apiError` pair, and it is covered in part 3.

1.5 Repository map

```
social/
├─ back/                          REST API
│  ├─ src/
│  │  ├─ env.js                   dotenv bootstrap + validated config
│  │  ├─ index.js                 connect, listen, graceful shutdown
│  │  ├─ app.js                   middleware chain, routes, error handler
│  │  ├─ db/index.js              mongoose.connect
│  │  ├─ redis/redis.js           Redis client + in-memory fallback
│  │  ├─ models/                  User, Video, Subscription, Message
│  │  ├─ controllers/             user, video, subscription, socket, library
│  │  ├─ routes/                  three routers
│  │  ├─ middlewares/             auth, cache, multer, ratelimiter, pagination
│  │  └─ utils/                   apiError, apiResponse, asyncHandler,
│  │                              cloudinary, cookies, elasticsearch
│  └─ script/seed-test-user.js    demo account
│
├─ SocketConn/                    Socket.IO presence service
│  ├─ src/server.js
│  ├─ database/index.js
│  └─ models/user.models.js
│
└─ frontend/
   └─ src/
      ├─ main.jsx                 providers (QueryClient, Router)
      ├─ App.jsx                  theme, socket registration, ToastHost
      ├─ routes/routes.jsx        route table, lazy chunks, auth gates
      ├─ api/                     axios instance + interceptors, endpoints
      ├─ components/              navbar, toast, loader, protected, boundary
      ├─ layout/                  MainLayout (with navbar) / AuthLayout (bare)
      ├─ pages/                   feed, watch, channel, messages, saved, …
      └─ utils/                   format helpers, theme map, video menu
```

Next: **02 — JavaScript the project depends on**

2 — The JavaScript This Project Depends On

> Language-level concepts, each anchored to real code in this repo.
> If you can explain every section here from the code, you can handle a JavaScript interview round.

2.1 ES modules: imports are hoisted and evaluated first

This caused the single most confusing bug in the codebase.

```
// back/src/index.js — THE ORIGINAL, BROKEN
import dotenv from "dotenv";
import { connectRedis } from "./redis/redis.js";   // ← evaluated NOW

dotenv.config({ path: "./.env" });                 // ← runs LATER
```

```
// back/src/redis/redis.js
const client = createClient({ url: process.env.REDIS_URL });  // undefined!
```

Why

An ES module is processed in three phases:

1. **Construction** — parse, resolve every `import` specifier, fetch the files.
2. **Instantiation** — allocate the bindings, wire exports to imports.
3. **Evaluation** — run the module bodies, **in depth-first post-order**.

Crucially, *every imported module body runs to completion before a single statement of the importing module runs*. Imports are not statements that execute in place. They are declarations, hoisted like `function`.

So the real order was:

```
redis.js body     → process.env.REDIS_URL is undefined → localhost:6379
app.js body       → cors({ origin: undefined })
cloudinary.js     → cloudinary.config({ cloud_name: undefined })
index.js body     → dotenv.config()      ← far too late
```

The symptom: `ECONNREFUSED 127.0.0.1:6379` even though `REDIS_URL` pointed at Upstash. `createClient` defaults to localhost when `url` is `undefined`.

The fix

Put `dotenv.config()` in its own module and import it **first**. Module bodies evaluate in source order, so this is deterministic:

```
// back/src/index.js — FIXED
import env, { assertRequiredEnv } from "./env.js";   // dotenv runs here
import { MongoConnection } from "./db/index.js";
import { connectRedis } from "./redis/redis.js";     // env is populated now
```

Why CommonJS didn't have this problem

```
const dotenv = require("dotenv");
dotenv.config();                        // runs HERE
const redis = require("./redis.js");    // runs AFTER
```

`require()` is a *function call* — synchronous, in place. `import` is a static declaration resolved before execution. This is the difference that lets ESM do tree-shaking and circular-import resolution, and it is the difference that bit us.

Interview answer: "ESM imports are hoisted and the dependency graph is evaluated depth-first before the importing module's body. So any module that reads `process.env` at the top level sees it before `dotenv.config()` runs unless you load dotenv in a module that is imported first — or use `node --env-file` / `node -r dotenv/config`."

2.2 Optional chaining stops at the `?.`, not after it

```
// BROKEN — back/src/controllers/user.controller.js
const avatarLocalPath     = req.files?.avatar[0]?.path;
const coverImageLocalPath = req.files?.coverImage[0]?.path;
```

`?.` only guards the link it sits on. `req.files?.avatar` is safe, but the `[0]` that follows is **not**. Cover image is optional, so registering without one threw:

```
TypeError: Cannot read properties of undefined (reading '0')
```

...a 500 on a perfectly valid request.

```
// FIXED
const avatarLocalPath     = req.files?.avatar?.[0]?.path;
const coverImageLocalPath = req.files?.coverImage?.[0]?.path;
```

Short-circuiting

When the left side is `null` / `undefined`, the **whole chain** short-circuits to `undefined` — the rest is never evaluated:

```
data?.user.avatar
// data == null      → undefined        (`.user.avatar` skipped entirely)
// data = {user:null} → TypeError!       (data is fine, so `.user` evaluates)
```

That second case is exactly why the navbar crashed for a user whose session had half-expired, and why it now reads `data?.user?.avatar`.

2.3 `??` vs `||`

```
// back/src/middlewares/cache.middleware.js
`user:${req.user?._id ?? "public"}`
```

	falls back on		
`\`	\	`	every falsy value: `0`, `""`, `false`, `NaN`, `null`, `undefined`
`??`	only `null` and `undefined`		

The bug this prevents, straight from the practice code:

```
if (minViews) filter.views.$gte = Number(minViews);   // ?minViews=0 ignored!
```

`0` is falsy, so "at least 0 views" silently disappeared. For numeric query parameters the test must be `!== undefined`:

```
if (minViews !== undefined) filter.views.$gte = Number(minViews);
```

`??` cannot be mixed with `&&` / `||` without parentheses — that is a syntax error by design, because the precedence would be ambiguous.

2.4 Closures — what the rate limiter and the cache middleware are

```
// back/src/middlewares/ratelimiter.middleware.js
export const rateLimiter = ({ limit = 30, windowSeconds = 20 } = {}) => {
    return async (req, res, next) => {
        //     ↑ this inner function still sees `limit` and `windowSeconds`
        //       after rateLimiter() has already returned
    };
};

export const rateLimit     = rateLimiter();
export const authRateLimit = rateLimiter({ limit: 10, windowSeconds: 60 });
```

A **closure** is a function plus the lexical environment it was created in.

`rateLimit` and `authRateLimit` are two separate function objects, each holding its own captured `limit`. This is the standard "middleware factory" pattern — `cache(300)` and `validatePagination({ maxLimit: 24 })` are the same idea.

Note `= {}` on the parameter: without it, calling `rateLimiter()` with no argument would destructure `undefined` and throw.

The classic closure interview question

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);  // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);  // 0 1 2
```

`var` is function-scoped: one binding, shared by all three closures, read after the loop finished. `let` is block-scoped and the spec re-binds it per iteration, so each closure captures its own `i`.

React's stale-closure bugs are this exact mechanism — see part 5.

2.5 `this` and why arrow functions break Mongoose hooks

```
// back/src/models/user.models.js
userSchema.pre("save", async function () {
    if (!this.isModified("password")) return;      // `this` = the document
    this.password = await bcrypt.hash(this.password, 10);
});

userSchema.methods.isPasswordCorrect = async function (password) {
    return await bcrypt.compare(password, this.password);
};
```

These **must** be `function` declarations. An arrow function has no `this` of its own — it closes over the `this` of the enclosing scope, which at module level is `undefined` in ESM. Mongoose calls the hook with the document as `this`, and an arrow function ignores that entirely.

`this` is determined by **how a function is called**:

Call form	`this`
`obj.fn()`	`obj`
`fn()`	`undefined` (strict / ESM)
`fn.call(x)` / `.apply` / `.bind(x)`	`x`
`new Fn()`	the new instance
arrow	lexical — inherited, never rebindable

2.6 Promises, async/await, and the two "swallowed error" bugs

`Promise.all` vs `Promise.allSettled`

```
// back/src/controllers/video.controller.js — two independent queries
const [videos, total] = await Promise.all([
    Video.find(filter).sort(sortOption).skip(skip).limit(limit).lean(),
    Video.countDocuments(filter),
]);
```

Running them sequentially would take `t1 + t2`. In parallel it is `max(t1, t2)`. They do not depend on each other, so there is no reason to wait.

`Promise.all` rejects as soon as **any** input rejects. For shutdown, where each cleanup should be attempted regardless, use `allSettled`:

```
// back/src/index.js
await Promise.allSettled([disconnectRedis(), MongoDisconnect()]);
```

	behaviour	
`all`	first rejection wins; others keep running but are ignored	
`allSettled`	waits for all; returns `{status, value | reason}[]`	
`race`	first to settle, fulfilled or rejected	
`any`	first to *fulfil*; rejects with `AggregateError` only if all reject	

Fire-and-forget, deliberately

```
// A view counter must never slow down or break playback.
Video.updateOne({ _id: video._id }, { $inc: { views: 1 } }).catch(() => {});
```

The `.catch()` is **mandatory**. An unhandled rejection in Node crashes the process by default from Node 15 onwards. The `void` keyword used elsewhere (`void indexVideo(videoDoc)`) documents "I am intentionally not awaiting this" — it does not handle errors, so the function itself must never reject.

The bug: throwing from a non-wrapped async handler

```
// BROKEN — this route HUNG instead of erroring
export const PracticeVideo = async (req, res) => {
    try { /* ... */ }
    catch (error) { throw new apiError(400, "Could'nt fetch"); }
};
```

Express 4 does not understand promises. When an `async` handler rejects, Express never sees it: no response is sent, the socket stays open, and the client waits until it times out. Meanwhile Node logs an unhandled rejection.

The fix is one word — wrap it:

```
export const PracticeVideo = asyncHandler(async (req, res) => { ... });
```

The other bug: `catch` that destroys the error

```
catch (error) {
    throw new apiError(401, "The buildVideoQuery helper went through an error");
}
```

The original error — a `CastError`, a timeout, a typo — is gone. Production gives you a meaningless 401 and no stack. Either let it propagate, or attach the cause:

```
catch (error) {
    throw new apiError(500, `Query build failed: ${error.message}`, [], error.stack);
}
```

2.7 The event loop (and why ordering questions get asked)

```
console.log("1");
setTimeout(() => console.log("2"), 0);     // macrotask
Promise.resolve().then(() => console.log("3"));  // microtask
queueMicrotask(() => console.log("4"));
console.log("5");
// 1 5 3 4 2
```

The rule: **the entire microtask queue is drained between every macrotask.**

Promise callbacks, `queueMicrotask` and `process.nextTick` are microtasks; timers, I/O and `setImmediate` are macrotasks.

Node's phases, in order: timers → pending callbacks → poll → check (`setImmediate`) → close callbacks. `process.nextTick` jumps the queue ahead of promises.

Practical consequence for this codebase: a long synchronous loop (a huge `JSON.stringify`, an unbounded `Video.find()` with no `.limit()`) blocks the *entire* server — Node is single-threaded for JavaScript. That is why the `Trending` endpoint returning the whole collection was a real availability risk, not just a slow query.

2.8 Spread, rest, and shallow copies

```
// Adding a derived field without mutating the source document
videos.docs = videos.docs.map((v) => ({
    ...v,
    durationFormatted: formatDuration(v.duration),
}));
```

`...` copies **one level**. Nested objects are shared by reference:

```
const a = { user: { name: "x" } };
const b = { ...a };
b.user.name = "y";
a.user.name;        // "y"  — same object!
```

For a real deep copy use `structuredClone(a)` (built into Node 17+ and modern browsers).

This matters in React: `setState({ ...state })` only triggers a re-render if the top-level reference changes, and only propagates a nested change if you copy that level too.

```
// frontend — correct nested update
setLoadedThumbs((prev) => (prev[id] ? prev : { ...prev, [id]: true }));
//                        ↑ returning the SAME reference skips the re-render
```

That guard is deliberate: returning `prev` unchanged tells React nothing happened, avoiding an infinite render loop from the `ref` callback.

2.9 `Object.hasOwn` and the mass-assignment fix

```
// back/src/controllers/video.controller.js
const UPDATABLE_FIELDS = ["title", "description", "isPublished"];

for (const field of UPDATABLE_FIELDS) {
    if (Object.hasOwn(req.body, field) && req.body[field] !== undefined) {
        video[field] = req.body[field];
    }
}
```

This replaced `Object.assign(video, req.body)`, which let a client send `{"views": 999999, "owner": "<someone else>"}` and have it written.

`Object.hasOwn(obj, key)` is the modern replacement for `Object.prototype.hasOwnProperty.call(obj, key)`. Plain `obj.hasOwnProperty(k)` is unsafe because an object can be created with `Object.create(null)` (no prototype, so no method) or can have `hasOwnProperty` overwritten by an attacker-controlled key.

It also beats `key in obj`, which walks the prototype chain — so

`"toString" in {}` is `true`.

2.10 Regex injection — user input is not a pattern

```
// BROKEN
{ title: { $regex: search, $options: "i" } }
```

A user typing `(((((` makes Mongo throw. A user typing `.*` or `(a+)+$` turns the query into a full collection scan or a catastrophic backtracking hang (ReDoS).

```
// FIXED
function escapeRegex(value = "") {
    return value.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
}
```

`$&` in a replacement string means "the whole match", so each metacharacter is prefixed with a backslash. Same principle as SQL parameterisation: **never concatenate user input into a language you then execute.**

2.11 `Intl` — formatting without a date library

```
// frontend/src/utils/format.jsx
new Intl.NumberFormat(undefined, { notation: "compact" }).format(1234);  // "1.2K"
new Intl.RelativeTimeFormat(undefined, { numeric: "auto" })
    .format(-3, "hour");                                                  // "3 hours ago"
```

Built into every modern browser and into Node. Zero bundle cost, and it follows the user's locale automatically. Reaching for `moment` (290 kB) or even `date-fns` to render "3 hours ago" is a bundle-size answer worth having ready.

`undefined` as the first argument means "use the runtime's default locale" — better than hardcoding `"en-US"`.

2.12 Labelled quick-reference

Concept	Where in this repo
ESM hoisting	`back/src/env.js`, `index.js`
Optional chaining depth	`registerUser`, `navbar.jsx`
`??` vs `||`	`||`	`cache.middleware.js`, numeric filters
Closures / factories	`rateLimiter`, `cache`, `validatePagination`
`this` binding	`user.models.js` hooks and methods
`Promise.all` / `allSettled`	`video.controller.js`, `index.js`
Unhandled rejection	`PracticeVideo` before the fix
Event loop	fire-and-forget view counter
Shallow copy	`setLoadedThumbs`, `videos.docs.map`
`Object.hasOwn`	`updateVideo` whitelist
Regex escaping	`escapeRegex` in `video.controller.js`
`Intl`	`utils/format.jsx`

Next: **03 — Node & Express**

3 — Node & Express

3.1 `asyncHandler` — the single most important 6 lines

```
// back/src/utils/asyncHandler.js
const asyncHandler = (requestHandler) => {
    return (req, res, next) => {
        Promise.resolve(requestHandler(req, res, next)).catch(next);
    };
};
```

The problem it solves

Express 4's router wraps each handler in a `try/catch`. That catches **synchronous** throws only:

```
app.get("/a", (req, res) => { throw new Error("caught"); });          // ✅ 500
app.get("/b", async (req, res) => { throw new Error("NOT caught"); }); // ❌ hangs
```

An `async` function never throws to its caller — it returns a rejected promise. Express ignores the return value, so the rejection is unobserved: no response is ever sent, the connection hangs until the client times out, and Node logs `UnhandledPromiseRejection`.

How the wrapper fixes it

`Promise.resolve(x)` normalises *anything* into a promise. `.catch(next)` forwards the rejection into Express's error pipeline, where the global error handler turns it into a JSON response.

`Promise.resolve` rather than just `.catch` on the call matters: it works whether the handler is `async`, returns a promise, or returns nothing at all.

Express 5 note

Express 5 (which this project uses — `"express": "^5.2.1"`) forwards rejected promises to `next` automatically. So `asyncHandler` is now belt-and-braces rather than strictly required. **Say this in an interview** — knowing *why* it existed and that v5 changed it is the interesting part.

The bug it would have prevented

```
// BROKEN: plain async, throws inside catch
export const Trending = async (req, res) => {
    try { /* ... */ }
    catch (error) { throw new apiError(401, "Cannot fetch"); }
};
```

The navbar's typeahead hits this route. Any error meant a hung request per keystroke. One word — `asyncHandler(...)` — fixed it.

3.2 The error pipeline

```
// back/src/utils/apiError.js
class apiError extends Error {
    constructor(statusCode, message = "Something Went Wrong", errors = [], stack) {
        super(message);
        this.statusCode = statusCode;
        this.data = null;
        this.success = false;
        this.errors = errors;

        if (stack) this.stack = stack;
        else Error.captureStackTrace(this, this.constructor);
    }
}
```

The bug in the original

```
constructor(statusCode, message, errors = [], stack = " ") {   // ← note the space
    ...
    if (stack) { this.stack = stack; }      // " " is TRUTHY
    else { Error.captureStackTrace(...); }
}
```

`" "` is a non-empty string, therefore truthy. **Every error in the entire application** had its stack trace replaced with a single space. Debugging anything in production would have been impossible.

Defaulting to `undefined` restores `Error.captureStackTrace`, which also omits the constructor frame so the trace starts at the actual throw site.

Error-handling middleware

```
// back/src/app.js
application.use((err, req, res, next) => { ... });
```

Express identifies an error handler **by arity** — a function with exactly four parameters. Three parameters makes it ordinary middleware, and your error handler silently never runs. You cannot drop the unused `next`.

It must also be registered **last**, after all routes, because Express walks the stack in registration order.

404s as errors

```
application.use((req, res, next) => {
    next(new apiError(404, `Route ${req.method} ${req.originalUrl} not found`));
});
```

A middleware with no path matches everything. Placed after all routes, it only runs when nothing matched. Routing it through `apiError` means a 404 has the same response shape as every other failure — the frontend never needs a special case.

3.3 Route ordering

```
// back/src/routes/socket.route.js
routerSocket.get("/convo", getConversations);
routerSocket.get("/conversation-id", getConversationId);
routerSocket.get("/:receiverId", getMessages);   // ← wildcard LAST
```

Express matches **top to bottom, first match wins**. The original had `/:receiverId` registered before `/generateConvoId`, so `GET /generateConvoId` was captured by the param route with `receiverId = "generateConvoId"` — the endpoint was unreachable.

Rule: static paths before dynamic ones. React Router v6+ is different — it *ranks* routes by specificity rather than using source order — but Express is strictly ordered.

The same file also had:

```
routerSocket.get("/generateConvoId", getConversationId);
```

where `getConversationId` was `(user1, user2) => ...` — a *utility*, not an Express handler. Called as `(req, res)` it would compute `[req, res].sort().join("_")` and return a string. Returning a value from a handler does nothing; the request hangs forever. Handlers must call `res.send` / `res.json` / `next`.

3.4 CORS with credentials

```
// back/src/app.js
cors({
    origin(origin, callback) {
        if (!origin) return callback(null, true);          // curl, same-origin
        if (env.CORS_ORIGINS.includes(origin)) return callback(null, true);
        return callback(new Error(`Origin ${origin} not allowed by CORS`));
    },
    credentials: true,
})
```

The rules you must know

1. `Access-Control-Allow-Origin: *` is **forbidden** with `Access-Control-Allow-Credentials: true`. The browser rejects the response. So the origin has to be echoed back from an allow-list.
1. The server must send `Access-Control-Allow-Credentials: true`, *and* the client must send `withCredentials: true`. Both, or cookies are dropped.
1. Non-simple requests (PATCH, DELETE, JSON content-type, custom headers) trigger a **preflight** `OPTIONS`. If preflight fails, the real request is never sent.
1. Requests with no `Origin` header (curl, server-to-server, same-origin) are not subject to CORS at all.

What was broken

```
cors({ origin: process.env.CORS_ORIGIN, credentials: true })
```

Because of the ESM hoisting bug (part 2.1), `process.env.CORS_ORIGIN` was

`undefined` at module-evaluation time. `cors` then produced no `Access-Control-Allow-Origin` header and every browser request failed.

Verify it from a terminal:

```
curl -i -X OPTIONS http://localhost:8000/api/v1/users/login -H "Origin: http://localhost:5173"
-H "Access-Control-Request-Method: POST"
```

You want `204` plus `Access-Control-Allow-Origin: http://localhost:5173` and `Access-Control-Allow-Credentials: true`.

CORS is enforced by the browser, not the server. It is not a security boundary against scripted clients — it protects *users* from other *sites* reading their authenticated responses.

3.5 File uploads: multer → Cloudinary → unlink

```
// back/src/middlewares/multer.middleware.js
filename(req, file, cb) {
    const ext = path.extname(file.originalname).toLowerCase().slice(0, 10);
    const safeExt = /^\.[a-z0-9]+$/.test(ext) ? ext : "";
    cb(null, `${Date.now()}-${crypto.randomBytes(8).toString("hex")}${safeExt}`);
}
```

Two bugs in the original

```
filename(req, file, cb) { cb(null, file.originalname); }
destination(req, file, cb) { cb(null, "./public"); }
```

1. **Collisions.** Two users uploading `video.mp4` at the same moment overwrite each other. User A gets User B's file.
1. **Path traversal.** `originalname` is attacker-controlled. A name like `../../src/app.js` can escape the directory. (Multer sanitises this in recent versions, but relying on that is not a defence you want to explain.)

Files also went to `./public`, which `express.static("public")` was serving — so every upload was briefly a publicly-readable URL on the API host.

Disk vs memory storage

	pros	cons
`diskStorage`	constant memory, handles 200 MB video	needs cleanup, needs disk
`memoryStorage`	no cleanup, simple	a 200 MB upload is 200 MB of RAM

Video uploads mean disk storage is the only sane choice here.

Guaranteed cleanup with `finally`

```
// back/src/utils/cloudinary.js
try {
    return await cloudinary.uploader.upload(localfilepath, { resource_type: "auto", folder });
} catch (error) {
    throw new Error(`Cloudinary upload failed: ${error.message}`);
} finally {
    await removeLocal(localfilepath);     // runs on BOTH paths
}
```

The original called `fs.unlinkSync` in both the `try` and the `catch`. If the file was already gone, the `catch` block's own unlink threw `ENOENT`, masking the real upload error. `finally` + a `.catch(() => {})` on the unlink makes cleanup unconditional and silent.

It also now **throws** instead of returning `null`, so the caller can report *why* ("Cloudinary not configured") rather than a generic "upload failed".

3.6 Graceful shutdown

```
// back/src/index.js
async function shutdown(signal) {
    if (shuttingDown) return;            // SIGINT twice shouldn't double-run
    shuttingDown = true;

    server.close(async () => {           // stop accepting NEW connections,
        await Promise.allSettled([       // let in-flight ones finish
            disconnectRedis(),
            MongoDisconnect(),
        ]);
        process.exit(0);
    });

    setTimeout(() => process.exit(1), 10000).unref();   // hard cap
}

process.on("SIGINT",  () => shutdown("SIGINT"));
process.on("SIGTERM", () => shutdown("SIGTERM"));
```

The original was `process.on("SIGINT", () => process.exit())` — instant exit, killing in-flight requests and leaving sockets to Mongo and Redis open.

- `server.close(cb)` stops new connections; the callback fires once existing ones drain.
- `.unref()` on the timeout means that timer alone will not keep the process alive — if everything closes in 200 ms, we exit in 200 ms.
- **SIGTERM is what Docker/Kubernetes/Render send.** Handling only SIGINT means your container is `SIGKILL`ed after the grace period.

3.7 Middleware order in `app.js`

```
application.use(cors(...));                       // 1 — before anything that can fail
application.use(express.json({ limit: "16kb" })); // 2 — parse JSON bodies
application.use(express.urlencoded({ extended: true, limit: "16kb" }));
application.use(cookieParser());                  // 3 — req.cookies

application.get("/health", ...);                  // 4 — cheap, unauthenticated

application.use("/api/v1/users", router);         // 5 — routes
application.use("/api/v1/videos", routerVideo);
application.use("/api/v1/socket", routerSocket);

application.use(notFoundHandler);                 // 6 — nothing matched
application.use(errorHandler);                    // 7 — four args, LAST
```

Order is semantics, not style:

- **CORS first.** An error thrown before CORS runs produces a response with no CORS headers, and the browser reports it as a generic "network error" rather than your actual 500.
- **Body parsers before routes**, or `req.body` is `undefined`.
- `limit: "16kb"` caps JSON bodies — the default is 100 kb, and an unbounded parser is a trivial memory-exhaustion vector. Files bypass this entirely because multer handles `multipart/form-data`.
- `express.static("public")` was **removed**: it was serving the upload temp directory.
- `app.set("trust proxy", 1)` was added so `req.ip` is the real client IP behind Render's load balancer — otherwise the rate limiter buckets every user under the proxy's single IP.

3.8 Config validation at boot

```
// back/src/env.js
export function assertRequiredEnv() {
    const required = [
        ["MONGODB_URL", env.MONGODB_URL],
        ["ACCESS_TOKEN_SECURITY", env.ACCESS_TOKEN_SECRET],
        ["REFRESH_TOKEN_SECURITY", env.REFRESH_TOKEN_SECRET],
    ];
    const missing = required.filter(([, v]) => !v).map(([n]) => n);
    if (missing.length) {
        console.error(`[env] Missing required variables: ${missing.join(", ")}`);
        process.exit(1);
    }
}
```

Fail fast. Without this, a missing `ACCESS_TOKEN_SECURITY` means `jwt.sign(payload, undefined)` throws on the *first login attempt* — a 500 in production, hours after deploy, with a confusing message. Crashing at boot means the deploy fails and the old version keeps serving.

Note also that only the genuinely-required variables are checked. Redis and Elasticsearch are optional by design (part 7), so they must not be in this list.

Next: **04 — MongoDB & Mongoose**

4 — MongoDB & Mongoose

4.1 The data model

```
User                        Video                     Subscription
──────────                  ──────────                ──────────────

_id                         _id                       _id
username   (unique, idx)    title                     subscriber ─┐ unique
email      (unique)         description               channel    ─┘ compound
fullName                    videoFile  (Cloudinary)
avatar                      thumbnail
coverImage                  duration   (seconds)      Message
password   (bcrypt)         views                     ──────────────
refreshToken                isPublished               _id
watchHistory  [Video]       owner ──────► User        sender    ──► User
watchLater    [Video]       createdAt                 receiver  ──► User
notInterested [Video]                                 conversationId (idx)
isOnline                                              text
lastSeen                                              messageType
                                                      status (sent|seen)
```

Embed or reference?

Relation	Choice	Why
Video → owner	**reference**	A user has unbounded videos; embedding would duplicate the profile into every video and make an avatar change a mass update.
User → watchHistory	**reference array, capped at 100**	Bounded by `$slice`, read with the user, never queried in reverse.
User → watchLater	**reference array**	Small, personal, always read with the user. Nobody asks "who saved this video?".
Subscription	**separate collection**	A many-to-many that must be queried from *both* directions (my subs, my subscribers) and counted.
Message	**separate collection**	Unbounded growth. Embedding would hit the 16 MB document limit.

The 16 MB document limit is the hard constraint behind all of this. Any unbounded list must be its own collection, or capped.

Why `conversationId` is derived, not stored

```
export const buildConversationId = (user1, user2) =>
    [String(user1), String(user2)].sort().join("_");
```

Sorting makes it symmetric: A→B and B→A produce the same key. So there is no `Conversation` collection to create, look up, or keep in sync — the key is a pure function of the participants. One indexed field answers "give me this thread".

The trade-off: it does not generalise to group chats. For those you *would* need a real `Conversation` document with a participants array. Worth saying out loud in an interview — it shows you know the limit of your own design.

4.2 Indexes

```
// back/src/models/video.models.js
videoSchema.index({ isPublished: 1, createdAt: -1 });   // the feed
videoSchema.index({ isPublished: 1, views: -1 });       // trending
videoSchema.index({ owner: 1, createdAt: -1 });         // channel page
videoSchema.index({ title: "text", description: "text" });

// back/src/models/subscription.models.js
subscriptionSchema.index({ subscriber: 1, channel: 1 }, { unique: true });
subscriptionSchema.index({ channel: 1 });
```

The ESR rule

Compound index field order: **Equality, Sort, Range.**

The feed query is `{ isPublished: true }` sorted by `createdAt: -1`.

`isPublished` is the equality match, `createdAt` is the sort → index `{ isPublished: 1, createdAt: -1 }`. Getting the order wrong means Mongo must fetch and sort in memory, and an in-memory sort over 32 MB **fails outright**.

A compound index also serves any **prefix** of itself:

`{a:1, b:1, c:1}` covers `{a}` and `{a,b}` — but not `{b}` or `{b,c}`.

Why the unique index on Subscription matters

Without it, a double-click on Subscribe creates two documents and `$size: "$subscribers"` is permanently wrong. Application-level

"check-then-insert" cannot prevent this — two requests can both read "not subscribed" before either writes. **Only the database can enforce it**, which is why the controller catches error code `11000`:

```
try {
    await Subscription.create(filter);
    subscribed = true;
} catch (err) {
    if (err?.code !== 11000) throw err;   // duplicate key — someone won the race
    subscribed = true;
}
```

That is a textbook check-then-act race, and the textbook fix.

4.3 The aggregation pipeline

```
// back/src/controllers/socket.controller.js — the inbox
Message.aggregate([
    { $match:  { $or: [{ sender: userId }, { receiver: userId }] } },
    { $sort:   { createdAt: -1 } },
    { $group:  { _id: "$conversationId",
                 lastMessage: { $first: "$$ROOT" },
                 unreadCount: { $sum: { $cond: [ ... , 1, 0] } } } },
    { $addFields: { otherUser: { $cond: [...] } } },
    { $lookup: { from: "users", localField: "otherUser", foreignField: "_id",
                 as: "otherUser",
                 pipeline: [{ $project: { username: 1, avatar: 1, ... } }] } },
    { $unwind: "$otherUser" },
    { $project: { ... } },
    { $sort:   { createdAt: -1 } },
])
```

Stage by stage:

Stage	Does
`$match`	filter — **put it first** so it can use an index
`$sort` + `$group` + `$first`	the "latest per group" idiom
`$cond` inside `$sum`	conditional counting — 1 when unread, else 0
`$addFields`	computed field, keeps all existing ones (`$project` does not)
`$lookup`	left outer join
`$unwind`	array → one document per element

Three rules worth memorising

1. `$match` **as early as possible.** Only a `$match` at the very start can use an index; after `$group` the documents are synthetic.
1. `$lookup` **always returns an array**, even for a one-to-one join. Flatten it with `$unwind` or `{ $first: "$field" }`.
1. `$project` **inside the** `$lookup` **sub-pipeline**, not after. Projecting afterwards means the full joined documents (including `password` and `refreshToken`) were already pulled into memory.

`$unwind` silently drops documents

```
{ $unwind: { path: "$owner", preserveNullAndEmptyArrays: true } }
```

Default `$unwind` removes any document whose array is empty. In the feed, a video whose owner row was deleted would **vanish from the feed with no error**. `preserveNullAndEmptyArrays: true` keeps it. This is one of the most common real-world aggregation bugs.

The bug: `pipeline` must be an array

```
// BROKEN — back/src/controllers/user.controller.js
$lookup: {
    from: "videos", localField: "watchHistory", foreignField: "_id",
    as: "watchhistory",
    pipeline: { $lookup: { ... } }        // ← an OBJECT
}
```

`pipeline` takes an **array of stages**. Passing an object made the whole watch-history endpoint error out. The nested `$addFields` was also misplaced *inside* the inner `$project` pipeline, where it could not see `$owner`.

```
// FIXED
pipeline: [
    { $lookup: { from: "users", localField: "owner", foreignField: "_id",
                 as: "owner",
                 pipeline: [{ $project: { username: 1, fullName: 1, avatar: 1 } }] } },
    { $addFields: { owner: { $first: "$owner" } } },
]
```

4.4 Pagination: skip/limit vs cursor

```
Video.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit)
```

`skip(n)` **walks and discards** n documents. At page 1000 with limit 20 the database reads 20,020 documents to return 20. It is O(n) in the offset.

There is a second, subtler problem: if a new video is inserted between the user loading page 1 and page 2, everything shifts down and they see the same item twice.

Cursor (keyset) pagination fixes both:

```
// Instead of ?page=3, pass the last item you saw:
Video.find({ ...filter, createdAt: { $lt: new Date(req.query.before) } })
     .sort({ createdAt: -1 })
     .limit(limit)
```

O(log n) via the index, and stable under inserts. The cost is that you lose "jump to page 7" — which is exactly the right trade for an infinite-scroll feed. The chat history already uses this shape (`?before=<ISO date>`).

The unstable-sort bug

```
switch (sort) {
    case "latest": sortOption = { createdAt: -1 }; break;
    case "oldest": sortOption = { createdAt:  1 }; break;
    // no default → sortOption = {}
}
```

An unrecognised `?sort=` value produced `{}` — "natural order", which MongoDB does **not** guarantee to be consistent between queries. Combined with skip/limit, that means duplicated and missing rows across pages. The fix

validates against an allow-list and always falls back to `latest`:

```
const sort = SORT_MAP[req.query.sort] ? req.query.sort : "latest";
```

4.5 Mongoose hooks — and the one I got wrong

```
// back/src/models/user.models.js
userSchema.pre("save", async function () {
    if (!this.isModified("password")) return;
    this.password = await bcrypt.hash(this.password, 10);
});
```

In Mongoose 9 an `async` pre-hook receives NO `next` callback. The returned promise *is* the completion signal — resolving means continue, throwing means abort the save. I initially "fixed" this by adding `next()`, and it failed immediately with:

```
TypeError: next is not a function
```

So the original `return` was correct. The lesson: a non-async hook takes `next`; an async hook returns a promise. Never mix them.

Why `isModified` is load-bearing

`generateAccessAndRefreshTokens` calls `user.save()` to store a new refresh token. Without the `isModified("password")` guard, *that* save would re-hash the already-hashed password — and the user could never log in again. One line standing between working auth and a permanently locked account.

`save()` vs `findByIdAndUpdate()`

```
user.password = newPassword;
await user.save();                 // ✅ hook runs → hashed
```

```
await User.findByIdAndUpdate(id, { password: newPassword });   // ❌ no hook
```

Update operators talk to the database directly; document middleware never runs. **A plaintext password would be stored.** This single distinction is a very common interview question, and a very common real breach.

Note the deliberate exception:

```
await user.save({ validateBeforeSave: false });   // storing refreshToken only
```

Skipping validation here avoids re-running required-field validators for a write that touches one unrelated field.

4.6 `.lean()`

```
Video.find(filter).limit(limit).lean()
```

By default Mongoose hydrates every result into a full document with getters, setters, virtuals and change tracking. `.lean()` returns plain objects instead — typically **3–5× faster and far less memory**.
Use it for read-only responses. Do *not* use it when you need `.save()`, a virtual, or an instance method like `isPasswordCorrect`.
Note `getVideoById` deliberately does **not** use `.lean()`, because it calls `video.toObject()` and relies on `populate` behaviour.

4.7 Atomic operators

```
Video.updateOne({ _id }, { $inc: { views: 1 } });
```

Never read-modify-write a counter:

```
const v = await Video.findById(id);
v.views += 1;            // ← two concurrent requests both read 10
await v.save();          //   both write 11; one view is lost
```

`$inc` is applied atomically by the server.

```
// Most-recent-first watch history, capped at 100
User.updateOne({ _id }, { $pull: { watchHistory: videoId } })
User.updateOne({ _id }, { $push: { watchHistory:
    { $each: [videoId], $position: 0, $slice: 100 } } })
```

`$pull` then `$push` because MongoDB cannot do both to one array in a single update. `$position: 0` puts it at the front; `$slice: 100` caps the array so

it can never grow unbounded.

```
$addToSet   // insert only if absent — IDEMPOTENT, so double-clicks are safe
$push       // always appends — duplicates possible
$unset      // removes the field entirely
$set        // sets it, including to null
```

The logout controller uses `$unset: { refreshToken: 1 }` rather than `$set: { refreshToken: undefined }` — the latter is a no-op in some driver versions, leaving the old refresh token valid after "logging out".

Next: **05 — React & the frontend**

5 — React & The Frontend

5.1 Rendering, reconciliation, keys

React re-renders a component when its state or props change. It builds a new element tree, **diffs** it against the previous one, and applies the minimal set of real DOM mutations.

The diff uses two heuristics:

1. Different element **type** at the same position → destroy the subtree and rebuild it.
1. Children are matched by **key**.

```
{videos.map((video) => (
    <div key={video._id}>…</div>
))}
```

Why `key={index}` is a bug

With index keys, deleting the first item renumbers everything. React thinks item 0 "changed from A to B" rather than "A was removed", so it **reuses the DOM node and its component state**. An input's typed value, a playing `<video>`, a half-finished animation — all attach to the wrong row.

Use a stable id from the data. Index keys are only safe for a list that is append-only and never reordered or filtered.

Note the toast host:

```
const id = `${Date.now()}-${Math.random().toString(36).slice(2, 7)}`;
```

`Date.now()` alone collides when two toasts fire in the same millisecond, and React warns about duplicate keys.

5.2 The hooks rules, and why

1. **Only call hooks at the top level** — never in a condition, loop or nested function.

1. **Only call hooks from React functions.**

React stores hook state in a **linked list per component instance**, indexed by call order. There are no names — the first `useState` is slot 0, the second is slot 1. Skip one conditionally and every subsequent hook reads the wrong slot.

```
// ❌ breaks the slot order
if (loading) { const [x] = useState(0); }
```

This is also why `Protected` returns *after* all its hooks have run — an early `return` before a hook would change the count between renders.

5.3 `useEffect` dependencies and stale closures

```
// frontend/src/components/navbar.jsx — debounce
useEffect(() => {
    const timer = setTimeout(() => setDebouncedTerm(searchTerm.trim()), 350);
    return () => clearTimeout(timer);
}, [searchTerm]);
```

The **cleanup function runs before the next effect** and on unmount. So each keystroke cancels the pending timer and starts a new one: the query fires only once, 350 ms after typing stops.

Without it, every keystroke fired a request — and the server's rate limiter (10 requests / 20 s at the time) returned 429 after about ten characters.

Debounce vs throttle

	fires
debounce	once, T after activity *stops* — search-as-you-type
throttle	at most once every T *during* activity — scroll, resize

The stale-closure trap

```
useEffect(() => {
    const id = setInterval(() => console.log(count), 1000);
    return () => clearInterval(id);
}, []);        // ← `count` is captured once, at mount, forever
```

The effect closes over `count` from the first render. Fixes: add `count` to the deps, or use the functional updater `setCount(c => c + 1)`, which never

needs to read the current value.

```
// This is why the thumbnail tracker uses the updater form:
setLoadedThumbs((prev) => (prev[id] ? prev : { ...prev, [id]: true }));
```

Returning `prev` **unchanged** when nothing happened is also deliberate — React bails out of the re-render when the reference is identical, which stops the `ref` callback from triggering an infinite loop.

`useEffectEvent` — a real bug in this repo

```
import { useState, useEffect, useEffectEvent } from "react";   // ❌
```

`useEffectEvent` is **experimental** and is not exported from the stable React build. Because React ships as CommonJS and Vite's interop does not throw on a missing named export, this silently evaluated to `undefined`. It would only have blown up at the call site. Removed.

5.4 `useRef` — two different jobs

```
const loadMoreRef = useRef(null);   // 1. a DOM handle
const toasted     = useRef(false);  // 2. mutable value that must NOT re-render
```

A ref is a stable box whose `.current` can change without triggering a render.

```
// frontend/src/components/protected.jsx
if (!checking && !user && !toasted.current) {
    toasted.current = true;
    toast("Log in to open that page");
}
```

Using `useState` here would re-render on every toast, which would re-run the effect, which would toast again. The ref breaks that cycle.

The infinite-scroll bug

```
// BROKEN
{videos.map((video) => (
    <div key={video._id} ref={loadMoreRef}>…</div>      // ref on EVERY card
))}
```

A ref holds one value. After rendering, `loadMoreRef.current` pointed at whichever card React committed last — and crucially, reassigning `.current` does **not** re-run the effect that called `observer.observe()`. So the observer kept watching a node that had already scrolled far out of view, and loading page 3 onwards was unreliable.

```
// FIXED — one sentinel below the grid
{videos.map(...)}
<div ref={loadMoreRef} className="h-px w-full" aria-hidden="true" />
```

`IntersectionObserver`

```
const observer = new IntersectionObserver(
    ([entry]) => {
        if (entry.isIntersecting && hasNextPage && !isFetchingNextPage) fetchNextPage();
    },
    { root: scrollContainer, threshold: 0.1, rootMargin: "400px" }
);
```

`rootMargin: "400px"` inflates the viewport, so the next page starts loading 400 px *before* the sentinel is visible — the user never sees the spinner. Far cheaper than a scroll listener, because the callback fires only on threshold crossings instead of on every scroll frame. Always `unobserve` / `disconnect` in the cleanup, or you leak the observer and the detached node it holds.

5.5 TanStack Query — server state is not UI state

```
const { data } = useQuery({
    queryKey: ["currentUser"],
    queryFn: getCurrentUser,
    staleTime: 1000 * 60,
    retry: false,
});
```

The core idea: data that lives on a server is **not** component state. It is a cache with freshness rules. Query gives you deduplication, caching, background refetch, and loading/error states for free.

Option	Meaning
`queryKey`	cache identity. Same key across components = **one** request
`staleTime`	how long data is considered fresh (no refetch)
`gcTime`	how long unused data stays in memory
`retry: false`	don't retry — a 401 means "not logged in", not "try again"
`placeholderData: prev => prev`	keep the old list visible during the next fetch

The navbar, `App`, and `Protected` all use `["currentUser"]`. Because the key matches, that is **one** network request shared by three components — not three.

`invalidateQueries` after a mutation

```
// frontend/src/pages/saved.jsx
const remove = useMutation({
    mutationFn: toggleWatchLater,
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["watchLater"] });
        queryClient.invalidateQueries({ queryKey: ["library"] });
    },
});
```

Both must be invalidated. `watchLater` drives this page's list; `library` drives the feed's "Remove from Watch later" label. Forgetting the second one leaves the two views disagreeing — a classic cache-coherence bug.

The same reasoning applies after login:

```
await queryClient.invalidateQueries({ queryKey: ["currentUser"] });
navigate(location.state?.from || "/", { replace: true });
```

Without it, the cached *signed-out* result is still fresh for another minute, so the navbar and `Protected` would both still believe nobody is logged in.

Optimistic updates

```
function handleNotInterested(video) {
    setHiddenIds((prev) => new Set(prev).add(video._id));   // hide immediately
    toast("You won't see this again");
    notInterestedMutation.mutate(video._id);
}
```

with a rollback when the server disagrees:

```
onError: (err, videoId) => {
    setHiddenIds((prev) => { const next = new Set(prev); next.delete(videoId); return next; });
}
```

The card disappears instantly instead of after a round trip. **Optimistic UI without a rollback path is just a lie** — the rollback is the non-optional half.

5.6 `useInfiniteQuery`

```
const { data, fetchNextPage, hasNextPage, isFetchingNextPage, isLoading } =
    useInfiniteQuery({
        queryKey: ["videos"],
        queryFn: fetchVideos,
        getNextPageParam: (lastPage) =>
            lastPage.page < lastPage.totalPages ? lastPage.page + 1 : undefined,
    });
```

`getNextPageParam` returning `undefined` is what sets `hasNextPage` to false.

`data.pages` is an array of page results, flattened for rendering:

```
data?.pages?.flatMap((page) => page.docs)
```

5.7 Code splitting — 330 kB → 78 kB gzip

```
const Login       = lazy(() => import("../auth/login"));
const Register    = lazy(() => import("../auth/register"));
const Messages    = lazy(() => import("../pages/message.jsx"));
const MessageList = lazy(() => import("../pages/messageList.jsx"));
```

`lazy()` takes a function returning a dynamic `import()`. The bundler sees the dynamic import and emits a **separate chunk**, fetched on first navigation.

`Suspense` is mandatory around lazy components — while the chunk is in flight the component suspends, and with no boundary that propagates as an error.

```
<Suspense fallback={<BlockLoader darkMode={darkMode} text="Loading" />}>
```

The bigger win: dead imports

The build was 1,207 kB. Two unused imports accounted for most of it:

```
import Aurora from "../assets/aurora.jsx";       // never rendered → pulls in `ogl`
import { GridScan } from "../assets/gridscan.jsx"; // never rendered → three.js,
                                                   // face-api.js, postprocessing
```

A static `import` runs the module and bundles it whether or not you use the binding. Tree-shaking cannot help: these modules have side effects and the bundler must assume they matter. Roughly 600 kB of JavaScript was downloaded by every visitor for two components that were never on screen.

	before	**after**
main chunk	1,207 kB (330 kB gz)	**273 kB (80 kB gz)**
socket.io	in main	own chunk, 188 kB
login / register / chat	in main	5–6 kB chunks each

Interview-ready claim: "I cut the initial JS payload by about 4× — mostly by deleting two imports that were never rendered, then route-level code splitting for the rest."

5.8 Error boundaries

```
export default class ErrorBoundary extends Component {
    static getDerivedStateFromError(error) { return { error }; }      // render phase
    componentDidCatch(error, info) { console.error(error, info); }    // commit phase
}
```

Must be a class — there is no hook equivalent, because these two lifecycle methods have no hook API.

Catches	**Does not catch**
render	event handlers
lifecycle methods	`setTimeout` / promises
child constructors	SSR, and errors in the boundary itself

Why it was added: `videos.map is not a function` on the watch page unmounted the **entire** React tree and left a blank white screen. With a boundary, the failure is contained and the user gets a "Try again" button.

5.9 The axios interceptor: single-flight refresh + circuit breaker

```
// frontend/src/api/api.jsx
let refreshPromise = null;
let refreshBlockedUntil = 0;
const REFRESH_COOLDOWN_MS = 30_000;
```

Three distinct problems, three mechanisms:

1. Thundering herd. Ten requests 401 at once. Without sharing, each fires its own refresh, each rotates the token, and nine are invalidated by the tenth — logging the user out.

```
if (!refreshPromise) {
    refreshPromise = api.post("/users/refreshtoken")
        .finally(() => { refreshPromise = null; });
}
await refreshPromise;
```

`.finally` rather than `.then` so a *failed* refresh also clears the slot — otherwise every later request awaits a permanently-rejected promise.

2. Infinite loop. The refresh call itself 401s → triggers a refresh → repeat.

```
const isAuthRoute = ["/users/refreshtoken", "/users/login", "/users/register"]
    .some((route) => url.includes(route));
```

plus `originalRequest._retry` so each request is retried at most once.

3. Pointless storms. An anonymous visitor produced ~90 requests per page: every page calls `/currentuser`, every 401 triggers a refresh, the refresh 401s because there is no session, and the next navigation repeats it.

```
catch (refreshError) {
    refreshBlockedUntil = Date.now() + REFRESH_COOLDOWN_MS;   // trip the breaker
    window.dispatchEvent(new Event("logout"));
}
```

A **circuit breaker**: once we know there is no session, stop asking for 30 seconds. `resetAuthState()` clears it on successful login.

5.10 Images: the pop-in fix

Three separate causes, worth knowing individually.

```
<img loading="lazy" decoding="async" fetchPriority="low" />   // ❌
```

1. `fetchPriority="low"` told the browser to deprioritise every thumbnail, so they arrived late while scrolling. `loading="lazy"` alone already defers off-screen images, and the browser starts them ~1250 px ahead.
1. **The `<img>` was unmounted** when the hover preview started and remounted on mouse-out, so the thumbnail had to decode again from scratch. It now stays mounted and the `<video>` layers on top.
1. **No placeholder** — an undecoded image showed as a hard black rectangle that snapped to the picture. Now it fades in over a shimmer.

The cached-image race

```
<img
    ref={(node) => { if (node?.complete && node.naturalWidth > 0) markThumbLoaded(id); }}
    onLoad={() => markThumbLoaded(id)}
    onError={() => markThumbLoaded(id)}
/>
```

An image served from the HTTP cache can finish loading **before React attaches the `onLoad` handler**. The event never fires and the element stays at `opacity-0` forever. `complete && naturalWidth > 0` is the reliable check — `complete` alone is also true for a *failed* load.

`onError` is there so a broken URL doesn't leave a permanent shimmer.

Layout shift

```
<div className="relative flex h-[102px] items-start gap-3 px-2 py-3">
```

The meta block has a fixed height and the views/age row is `absolute bottom-3`. CSS grid rows size to their tallest item, so without a reservation a one-line title made that card shorter and the whole row shifted as data streamed in. The skeleton uses the identical `h-[102px]`, so swapping placeholder for real content causes **zero** layout shift.

5.11 Where `localStorage` goes wrong

```
const [darkMode, setDarkMode] = useState(() => {
    try {
        return JSON.parse(localStorage.getItem("darkMode")) ?? false;
    } catch {
        return false;
    }
});
```

Two failure modes the original ignored:

1. `JSON.parse(null)` is `null` (fine), but `JSON.parse("undefined")`

throws — and a throw inside a `useState` initialiser blanks the entire app.

1. `localStorage` access itself throws in private mode or when site data is blocked.

```
function readStoredTheme() {
    const stored = localStorage.getItem("themeSelected");
    const known = Object.values(themeObject).flatMap((g) => Object.values(g));
    return known.includes(stored) ? stored : DEFAULT_THEME;
}
```

The persisted value was an **asset URL**, and Vite fingerprints asset URLs at build time (`/PCSmall1.jpg` in dev, `/assets/PCSmall1-a1b2c3.jpg` in prod). A URL stored before a deploy pointed at a 404 afterwards. Validating against the themes that currently exist makes it self-heal.

General rule: never persist a value whose meaning can change between builds. Persist a stable key and resolve it at runtime.

5.12 Lifting state vs prop drilling — an honest critique

`App.jsx` holds `menubar`, `darkMode`, `themeSelected`, `profileSelected`, `currentId`, `videoIdSelected`, `isLoggedIn` and passes all of them through `AppRoutes` into `MainLayout` into `Navbar`.

That is **prop drilling**, and it is the main thing a reviewer would flag.

The fix is Context — ideally split so unrelated updates do not re-render everything:

```
<ThemeContext.Provider value={{ darkMode, setDarkMode }}>
<UIContext.Provider value={{ menubar, setMenubar }}>
```

Separate contexts matter: **any** context value change re-renders every consumer, so one giant `AppContext` is barely better than drilling. Memoise the value object too, or a new object literal each render defeats the point. Note that `isLoggedIn` is now redundant — TanStack Query's `["currentUser"]` cache is the single source of truth, and `Protected` reads it directly. Keeping a duplicate boolean in `App` state is exactly the kind of derived state that drifts out of sync.

Next: **06 — Authentication**

6 — Authentication

This is the area with the highest density of interview questions, and it contained the single worst bug in the repository.

6.1 Password hashing

```
userSchema.pre("save", async function () {
    if (!this.isModified("password")) return;
    this.password = await bcrypt.hash(this.password, 10);
});

userSchema.methods.isPasswordCorrect = async function (password) {
    return await bcrypt.compare(password, this.password);
};
```

Hashing vs encryption

Encryption is reversible; hashing is not. You never need the original password back — you only need to check a candidate. So hashing is correct, and "we encrypt passwords" is a red flag in an interview answer.

Why bcrypt and not SHA-256

SHA-256 is designed to be *fast* — billions of guesses per second on a GPU. bcrypt is deliberately **slow and tunable**:

- **Cost factor 10** = 2^{10} = 1024 internal iterations. Each +1 doubles the work. ~100 ms per hash is the usual target: unnoticeable at login, brutal for an attacker.
- **Salt is automatic and per-user**, embedded in the output string:

```
$2b$10$N9qo8uLOickgx2ZMRZoMye  IjZAgcfl7p92ldGxad68LJZdL17lhWy
│  │  └── 22-char salt ──────┘  └──────────── hash ────────────┘
│  └──── cost 10
└─────── algorithm
```

A per-user salt means two users with the same password get different hashes, and a precomputed rainbow table is worthless.

- **Argon2id** is the modern recommendation (memory-hard, so it also resists GPU/ASIC attacks). bcrypt remains perfectly acceptable; scrypt is a third option. Never SHA/MD5 alone.

Timing attacks

`bcrypt.compare` is constant-time for a given hash. But note this in the login controller:

```
if (!user)             throw new apiError(401, "Invalid credentials");
if (!isPasswordValid)  throw new apiError(401, "Invalid credentials");
```

Same status, same message. The original said "User Not Found, you may register" vs "Invalid Password" — which lets anyone enumerate who has an account. A fully rigorous implementation would also hash a dummy password when the user doesn't exist, so the *response time* doesn't leak it either.

6.2 Access and refresh tokens

```
userSchema.methods.generateAccessToken = function () {
    return jwt.sign(
        { _id: this.id, username: this.username, email: this.email, fullName: this.fullName },
        process.env.ACCESS_TOKEN_SECURITY,
        { expiresIn: process.env.ACCESS_TOKEN_EXPIRY }
    );
};

userSchema.methods.generateRefreshToken = function () {
    return jwt.sign({ _id: this.id }, process.env.REFRESH_TOKEN_SECURITY,
                    { expiresIn: process.env.REFRESH_TOKEN_EXPIRY });
};
```

	access	**refresh**
lifetime	short (1 day here; 15 min is more typical)	long (10 days)
payload	id + profile fields	**id only**
stored server-side	no	**yes**, on the user document
sent with	every request	only to `/refreshtoken`

Why two tokens at all

A JWT cannot be revoked — it is valid until it expires, because verification

is pure signature maths with no database lookup. That is the whole point (it scales horizontally) and also the whole problem.

The compromise: make the *unrevocable* token short-lived, and gate renewal behind a long-lived token that **is** checked against the database:

```
if (incomingRefreshToken !== user.refreshToken) {
    throw new apiError(401, "Refresh token is expired or already used");
}
```

Logging out sets `$unset: { refreshToken: 1 }`, so the refresh token dies immediately. The access token survives at most one more expiry window.

Why the refresh payload is id-only

Payload is **base64-encoded, not encrypted** — anyone with the token can read it. Put nothing sensitive in it, and keep it small since it travels on every request.

```
atob("eyJfaWQiOiI2YWY0..."); // → {"_id":"6af4...","username":"demouser",...}
```

Rotation

Every successful refresh issues a **new** refresh token and overwrites the stored one. A stolen token is therefore single-use — and if both the attacker and the real user try to use it, the second one fails. (A production system would go further and treat that reuse as a signal to invalidate the entire token family.)

6.3 THE critical bug: `refreshAccessToken` was dead on arrival

```
// BROKEN — back/src/controllers/user.controller.js
const refreshAccessToken = asyncHandler(async (req, res) => {
    // ...
    res.cookie("refreshToken", token, {        // ← `token` is NOT DEFINED
        httpOnly: true,
        secure: isProd,
        sameSite: isProd ? "none" : "lax",
    });

    const { accessToken, refreshToken } = await generateAccessAndRefreshTokens(user._id);

    return res
        .status(200)
        .cookie("accessToken", accessToken, options)   // ← `options` is NOT DEFINED
        .cookie("refreshToken", refreshToken, options)
        .json(new apiResponse(200, {}, "access token refreshed"));
});
```

Two `ReferenceError`s. **Every single call threw a 500.**

The downstream effect

The frontend interceptor treats a failed refresh as "you are logged out":

```
catch (refreshError) {
    window.dispatchEvent(new Event("logout"));
}
```

So the moment an access token expired, the user was silently logged out and bounced to `/login` — with no error anyone could interpret. The "persistent login sessions" feature in the README did not work at all.

Verifying the fix

```
curl -c ck -X POST localhost:8000/api/v1/users/login \
     -H "Content-Type: application/json" \
     -d '{"username":"demouser","password":"demo1234"}'
```

```
curl -b ck -c ck -X POST localhost:8000/api/v1/users/refreshtoken
```

```
{"statusCode":200,"data":{},"message":"Access token refreshed","success":true}
```

Lesson: this is what a linter catches for free. `no-undef` would have flagged both identifiers before the code ever ran. The project has ESLint configured but it was clearly never run against the backend.

6.4 Cookies

```
// back/src/utils/cookies.js
const base = {
    httpOnly: true,
    secure: env.isProd,
    sameSite: env.isProd ? "none" : "lax",
    path: "/",
};

export const accessCookieOptions  = { ...base, maxAge: 1  * DAY };
export const refreshCookieOptions = { ...base, maxAge: 10 * DAY };
export const clearCookieOptions   = { ...base };
```

Flag	Why
`httpOnly`	JavaScript cannot read it → blunts XSS token theft
`secure`	HTTPS only. **Must be false on** `http://localhost` or the browser silently drops it
`sameSite`	CSRF control (below)
`maxAge`	matched to token lifetime, so cookie and JWT expire together
`path: "/"`	must match on `clearCookie` or logout silently fails

`sameSite` in detail

Value	Sent on cross-site requests?
`strict`	never — even following a link from another site
`lax`	only on top-level GET navigations
`none`	always — **requires** `secure: true`

Production runs the API on Render and the SPA on Netlify: different sites, so `none` + `secure` is mandatory. Locally both are `localhost`, so `lax` works and avoids the `secure` requirement over plain HTTP. Getting this wrong is the classic "works locally, broken in production" auth bug.

Why `clearCookieOptions` exists separately

`clearCookie` only removes a cookie if `path`, `domain`, `secure` and `sameSite` **match what was set**. A mismatch means logout appears to succeed and the cookie is still there. Deriving all three option sets from one `base` object makes that impossible to get wrong.

Cookies vs `localStorage` for JWTs

	cookie (`httpOnly`)	`localStorage`
XSS can read it	**no**	**yes** — one injected script takes everything
CSRF risk	yes → mitigated by `sameSite`	no
sent automatically	yes	no, you attach it manually

Cookies are the better default: XSS is more common than CSRF, and `sameSite` is a one-line CSRF mitigation whereas there is no mitigation for "a script can read your token".

6.5 `verifyJWT` and `optionalJWT`

```
export const verifyJWT = asyncHandler(async (req, res, next) => {
    const token = readToken(req);
    if (!token) throw new apiError(401, "Unauthorized request");

    let decodedToken;
    try {
        decodedToken = jwt.verify(token, env.ACCESS_TOKEN_SECRET);
    } catch (error) {
        const message = error?.name === "TokenExpiredError"
            ? "Access token expired" : "Invalid access token";
        throw new apiError(401, message);
    }

    const user = await User.findById(decodedToken?._id).select("-password -refreshToken");
    if (!user) throw new apiError(401, "Invalid access token");

    req.user = user;
    next();
});
```

Points worth making:

- `jwt.verify`, **never** `jwt.decode`. `decode` does not check the signature — an attacker can forge any payload.
- Distinguishing `TokenExpiredError` from a tampered token makes the network tab readable and tells the client whether a refresh is worth attempting.
- `.select("-password -refreshToken")` — `req.user` is passed around and serialised into responses; the password hash must never ride along.
- The database lookup on every request is the cost of being able to revoke. A higher-scale system would cache it in Redis with a short TTL.

```
export const optionalJWT = asyncHandler(async (req, res, next) => {
    const token = readToken(req);
    if (!token) return next();
    try { /* attach req.user if valid */ } catch { /* ignore */ }
    next();
});
```

Used for pages that render for everyone but show more when signed in — the channel page needs `isSubscribed`, and the watch page records watch history only if it knows who you are. Previously `/c/:username` required auth, so anonymous visitors could not view a channel at all.

6.6 Other auth bugs that were fixed

Query-object injection in login

```
const user = await User.findOne({ $or: [{ email }, { username }] });   // ❌
```

If the client posts only `username`, then `email` is `undefined` and the filter becomes `{ $or: [ { email: undefined }, { username: "x" } ] }`. MongoDB treats `{ field: undefined }` as "field does not exist" — so this can match an unrelated document that happens to have no `email`.

```
// ✅ build the $or from present values only
const or = [];
if (email?.trim())    or.push({ email: email.trim().toLowerCase() });
if (username?.trim()) or.push({ username: username.trim().toLowerCase() });
```

Note also the normalisation: the schema stores `lowercase: true`, so a lookup

with the raw input would miss `"DemoUser"`.

`getUserChannelProfile` read the wrong place

```
const { username } = req.body;        // ❌ route is GET /c/:username
```

A GET request has no body, so this **always** 404'd. Now it accepts both:

```
const username = (req.params.username ?? req.body?.username ?? "").trim().toLowerCase();
```

Avatar upload always returned 400

```
const avatarLocalPath = req.files?.path;    // ❌
```

multer's `.single(name)` populates `req.file` (singular). `.fields()` and `.array()` populate `req.files`. The route uses `.single("avatar")`, so `req.files` was always `undefined` and the endpoint could never succeed.

```
const localPath = req.file?.path;           // ✅
```

Rate limiting the credential endpoints

```
export const rateLimit     = rateLimiter();                              // 30 / 20s
export const authRateLimit = rateLimiter({ limit: 10, windowSeconds: 60 });

router.route("/login").post(authRateLimit, loginUser);
router.route("/register").post(authRateLimit, ...);
```

Login and register get a tighter budget than ordinary reads — this is the brute-force surface. A production system would additionally lock per-account after N failures, not just per-IP, since an attacker can rotate IPs.

6.7 The full flow, end to end

```
REGISTER
  validate → check existing → upload avatar → User.create
  → pre("save") hashes the password
  → issue access + refresh, set both cookies, 201
    (new: the user is logged in immediately instead of bounced to /login)

LOGIN
  find by email OR username → bcrypt.compare
  → issue access + refresh → store refresh on the user → set cookies

AUTHENTICATED REQUEST
  cookie → verifyJWT → jwt.verify → User.findById → req.user → handler

ACCESS TOKEN EXPIRES
  any request → 401
  → axios interceptor → single-flight POST /refreshtoken
  → verify refresh signature → compare against the stored one
  → issue a NEW pair (rotation) → retry the original request
  → on failure: trip the circuit breaker, emit "logout"

LOGOUT
  $unset refreshToken  (refresh dies now)
  clearCookie ×2 with matching options
  → frontend clears the ["currentUser"] cache
```

Next: **07 — Caching, scale & realtime**

7 — Caching, Rate Limiting, Realtime & Scale

7.1 Redis as a cache that cannot take you down

```
// back/src/redis/redis.js
const memoryStore = { get, set, setEx, del, keys, incr, expire };

async function withFallback(name, args) {
    if (ready && realClient) {
        try { return await realClient[name](...args); }
        catch (err) { console.warn(`[redis] ${name} failed, falling back`); }
    }
    return memoryStore[name](...args);
}
```

The design rule

Redis here is a cache and a counter, not a source of truth. Nothing in it is unrecoverable. So an outage must degrade performance, never availability.

The original did this:

```
await connectRedis();     // in index.js — throws if Redis is unreachable
```

A Redis hiccup took the entire API down. Now `connectRedis()` never throws, the client exposes one small surface, and every call silently falls back to an in-process `Map`.

The in-memory fallback is deliberately *correct but not distributed*: with multiple server instances each has its own map, so the cache hit rate drops and rate limits become per-instance. That is an acceptable degraded mode, and `/health` reports which mode you are in so it is never a silent surprise.

```
reconnectStrategy: (retries) => (retries > 5 ? false : Math.min(retries * 200, 2000))
```

Capped exponential-ish backoff, then give up — rather than retrying forever and filling the logs. We have a working fallback, so there is no reason to thrash.

7.2 Cache-aside

```
const cached = await redis.get(cacheKey);
if (cached) return res.status(200).json(JSON.parse(cached));   // HIT

const data = await expensiveQuery();                           // MISS
res.status(200).json(payload);                                 // respond FIRST
redis.set(cacheKey, JSON.stringify(payload), { EX: 300 })      // then cache
     .catch(...);
```

Pattern	Who writes the cache
cache-aside (used here)	the application, on a miss
read-through	the cache library itself
write-through	write to cache + DB together — consistent, slower writes
write-behind	write to cache, flush to DB later — fast, risks data loss

Two bugs in the original cache middleware

```
res.json = async (data) => {
    await redisClient.setEx(key, expiry, JSON.stringify(data));   // ❌ blocks
    originalJson(data);
};
```

1. **The client waited on Redis before receiving the body.** On a miss the user paid the database round trip *and* the Redis round trip. Now the response is sent first and the cache write is fire-and-forget.

1. **Error responses were cached.** The override caught every `res.json`, including 4xx and 5xx. One transient failure was then served to everyone for the full TTL.

```
if (res.statusCode >= 200 && res.statusCode < 300) {   // ✅ only cache success
    Promise.resolve(redisClient.setEx(key, expiry, JSON.stringify(data)))
           .catch(...);
}
```

Invalidation

```
export const invalidate = async (pattern) => {
    const keys = await redisClient.keys(pattern);
    if (keys.length) await redisClient.del(keys);
};

await invalidate("videos:*");   // after create / update / delete
```

`KEYS` **is O(n) over the entire keyspace and blocks Redis** — it is single threaded. Fine at this scale; in production you would use `SCAN` (cursor-based, non-blocking) or, better, a **generation counter**:

```
videos:gen7:page1   ← bump `gen` to 8 and every old key is orphaned,
                      then expires naturally. No scan at all.
```

Say this in an interview. "I used `KEYS` and here is why that doesn't scale, and here's what I'd do instead" is a much stronger answer than using `SCAN` without being able to explain either.

Key design

```
const key = [req.method, req.originalUrl, `user:${req.user?._id ?? "public"}`,
             JSON.stringify(req.body ?? {})].join(":");
```

The `user:` segment is a **correctness requirement**, not an optimisation. Without it, a personalised response cached for user A would be served to user B. That class of bug has leaked real users' private data in real products.

7.3 Rate limiting

```
const requests = await client.incr(key);
if (requests === 1) await client.expire(key, windowSeconds);
if (requests > limit) return res.status(429).json({ ... });
```

`INCR` on a missing key creates it at 1, atomically. The `=== 1` check sets the TTL exactly once, which is what makes it a *window*.

The boundary-burst flaw

Fixed-window counters allow **2× the limit** across a window boundary: 30

requests at t=19.9 s and another 30 at t=20.1 s is 60 requests in 200 ms.

Algorithm	Trade-off
fixed window (used here)	2 Redis ops, simple; boundary burst
sliding window log	exact; stores every timestamp — memory heavy
sliding window counter	weighted blend of two windows; good compromise
token bucket	allows deliberate bursts, smooth refill; the usual production choice

Fail-open vs fail-closed

```
catch (err) {
    console.warn(`[ratelimit] failing open: ${err.message}`);
    next();                                  // let it through
}
```

If the counter store is unreachable, we **fail open**. The alternative locks every user out of the whole product because a cache is down. For a rate limiter, availability beats strictness. (For an *authorisation* check the answer is the opposite — always fail closed.)

Identity

```
const identity = req.user?._id ? `user:${req.user._id}` : `ip:${req.ip}`;
```

Per-user when known, per-IP otherwise. And `app.set("trust proxy", 1)` is essential — behind Render's load balancer, `req.ip` without it is the *proxy's* IP, so every user in the world shares one bucket.

Headers

```
res.setHeader("X-RateLimit-Limit", limit);
res.setHeader("X-RateLimit-Remaining", Math.max(0, limit - requests));
res.setHeader("Retry-After", windowSeconds);     // on a 429
```

Tells a well-behaved client how to back off instead of hammering.

7.4 Realtime

```
// SocketConn/src/server.js
const onlineUsers = new Map();   // userId -> Set<socketId>

socket.on("register-user", async (userId) => {
    socket.userId = userId;
    socket.join(userId);                          // a room named after the user

    if (!onlineUsers.has(userId)) onlineUsers.set(userId, new Set());
    onlineUsers.get(userId).add(socket.id);

    if (onlineUsers.get(userId).size === 1) {    // first tab → now online
        await User.findByIdAndUpdate(userId, { isOnline: true, lastSeen: null });
        io.emit("user-status", { userId, isOnline: true });
    }
});
```

Why `Map<userId, Set<socketId>>` and not `Map<userId, socketId>`

One user, many tabs and devices. Tracking a **set** of sockets means closing one tab does not mark you offline while another is still open. The transition only fires when the set becomes empty:

```
sockets.delete(socket.id);
if (sockets.size === 0) {
    onlineUsers.delete(socket.userId);
    await User.findByIdAndUpdate(socket.userId, { isOnline: false, lastSeen: new Date() });
    io.emit("user-status", { userId: socket.userId, isOnline: false, lastSeen });
}
```

This is a reference-counting pattern, and it is the single most common thing people get wrong in a presence system.

Rooms

`socket.join(userId)` makes a room per user, so delivering a message is `io.to(receiverId).emit(...)` with no socket-id bookkeeping and automatic multi-device fan-out. `socket.join(conversationId)` does the same for typing indicators.

`socket.to(room)` excludes the sender; `io.to(room)` includes everyone. Typing indicators use `socket.to` — you should not see your own.

Known limitations (say these before you're asked)

1. **No authentication on the socket.** Anyone can emit `register-user` with any id and mark that user online, or join any conversation room. The fix is to verify the JWT in a handshake middleware:

```js
io.use((socket, next) => {
const token = parseCookie(socket.handshake.headers.cookie)?.accessToken;
try { socket.userId = jwt.verify(token, SECRET)._id; next(); }
catch { next(new Error("unauthorized")); }
});
```

and then **ignore** any client-supplied id.

1. `onlineUsers` **is in-process memory.** Two instances behind a load balancer each know only their own connections. Production needs the Socket.IO **Redis adapter** so rooms and broadcasts span instances, plus sticky sessions (or `transports: ["websocket"]`, which this client already forces) so the HTTP-polling handshake doesn't bounce between servers.

1. `transports: ["websocket"]` on the client skips the polling upgrade — faster, and avoids sticky-session problems — but fails outright on networks that block WebSocket rather than falling back.

Why messages go through REST, not the socket

The socket emits `send-message` for *delivery*, but the message is **created** by `POST /api/v1/socket/send`. So persistence, validation, rate limiting and auth all happen in one place, and a socket outage cannot lose data. The socket layer is a transport optimisation on top of a durable store.

7.5 Search: three tiers

```
export const searchElastic = asyncHandler(async (req, res, next) => {
    if (!elastic) { req.query.search = q; return Trending(req, res, next); }
    try { /* elastic multi_match with fuzziness */ }
    catch { return Trending(req, res, next); }      // degrade, don't fail
});
```

Tier	Cost	Quality
`$regex` (current default)	collection scan unless anchored	substring only, no ranking
Mongo `$text` index	indexed, stemming, `textScore`	no fuzziness, one text index per collection
Elasticsearch	separate cluster to run and sync	fuzzy, relevance, facets, highlighting

The honest engineering answer — and the one actually taken here — is that Elasticsearch is **optional**, defaults to off, and the code degrades to Mongo when it is absent. Running a cluster for a few thousand videos is not worth the operational cost.

A text index is already declared on the models as the sensible middle tier:

```
videoSchema.index({ title: "text", description: "text" });
```

Note the original `searchElastic` referenced `elastic` **without importing it** — a guaranteed `ReferenceError` had anything ever routed to it.

7.6 N+1 queries

```
// ❌ 1 + N round trips
const videos = await Video.find();
for (const v of videos) v.owner = await User.findById(v.owner);

// ✅ 1 round trip
const videos = await Video.find().populate("owner", "username fullName avatar");

// ✅ also 1, and composable with other stages
{ $lookup: { from: "users", localField: "owner", foreignField: "_id", as: "owner",
             pipeline: [{ $project: { username: 1, fullName: 1, avatar: 1 } }] } }
```

`populate` is a second query with `$in` (so 2 total, not N+1). `$lookup` is a true server-side join in one round trip. Either is fine; the loop is not.

Always project the fields you need. `populate("owner")` with no field list fetches the **password hash and refresh token** into memory — one careless `res.json(video)` away from a breach.

7.7 What would need to change at 100× traffic

Area	Now	At scale
Pagination	`skip` / `limit`	cursor/keyset — `skip` is O(offset)
Cache invalidation	`KEYS videos:*`	generation counter or `SCAN`
Rate limiting	fixed window	token bucket, per-user **and** per-IP
Sessions	DB lookup per request	Redis-cached user, short TTL
Presence	in-process `Map`	Socket.IO Redis adapter
Search	regex	`$text`, then Elasticsearch
Uploads	API proxies to Cloudinary	signed direct-to-Cloudinary upload
Feed	query per request	precomputed/fan-out timeline
Media	Cloudinary default	CDN + HLS/DASH adaptive bitrate
Deploy	single instance	horizontal + sticky or stateless sockets

The highest-value change by far is signed direct uploads. Right now a 200 MB video streams to the API, hits the disk, then streams to Cloudinary — the API server is tied up for the whole transfer and the disk is a shared, finite resource. A signed upload has the browser send bytes straight to Cloudinary and post only the resulting URL back. One change removes the largest scaling bottleneck in the system.

Next: **08 — Every bug, explained**

8 — Every Bug, Explained

Each entry: what it was, why it happened, the fix, and the transferable lesson. These make excellent "tell me about a bug you found" answers.

Severity: 🔴 broke a feature outright · 🟠 security/data · 🟡 correctness · 🔵 performance/UX

Backend

🔴 1. `refreshAccessToken` threw on every call

`back/src/controllers/user.controller.js`

```
res.cookie("refreshToken", token, { ... });          // `token` undefined
return res.status(200)
    .cookie("accessToken", accessToken, options)     // `options` undefined
```

Two `ReferenceError`s → 500 on every refresh. Because the axios interceptor treats a failed refresh as "logged out", **every session silently died the moment the access token expired.** The README's "persistent login sessions" never worked.

Lesson: ESLint's `no-undef` catches this for free. The project has ESLint configured for the frontend but it was never run against the backend.

🔴 2. `dotenv.config()` ran after every module had already read `process.env`

`back/src/index.js`

ESM imports are hoisted and evaluated before the importing module's body (see §2.1). So `redis.js`, `app.js` and `cloudinary.js` all read `process.env` while it was still empty:

- Redis connected to `localhost:6379` instead of Upstash → `ECONNREFUSED`
- `cors({ origin: undefined })` → no `Access-Control-Allow-Origin` header
- `cloudinary.config({ cloud_name: undefined })` → uploads broken

Fix: `back/src/env.js`, imported first.

Lesson: `import` is a hoisted declaration, not an in-place statement.

🔴 3. `updateUserAvatar` / `updateUserCoverImage` always returned 400

```
const avatarLocalPath = req.files?.path;     // ❌ always undefined
```

`multer.single(name)` populates `req.file`; only `.fields()` / `.array()` populate `req.files`. The endpoint could never succeed.

Lesson: `.single` → `req.file`. `.fields` / `.array` → `req.files`.

🔴 4. `getUserChannelProfile` read `req.body` on a GET route

```
const { username } = req.body;     // route is GET /c/:username
```

GET requests have no body → always 404. *Fix:* accept `req.params.username` with the body as a fallback.

🔴 5. `getWatchHistory`: `pipeline` must be an array

```
pipeline: { $lookup: { ... } }     // ❌ object where an array is required
```

Plus `user[0].watchhistory` with no guard (crash when the aggregation returned nothing) and a case mismatch against the `as:` alias.

🔴 6. Unreachable + non-functional socket routes

```
routerSocket.get("/:receiverId", getMessages);
routerSocket.get("/generateConvoId", getConversationId);   // never reached
```

Express matches in order, so `/:receiverId` swallowed it. And `getConversationId` was `(user1, user2) => string` — a utility, not a handler. Called as `(req, res)` it returned a string and never responded, hanging the request.

Lesson: static routes before dynamic ones; a handler must call `res` / `next`, never just return.

🔴 7. `PracticeVideo` / `Trending` hung instead of erroring

Plain `async` handlers that `throw` inside `catch`. Express 4 cannot see a rejected promise, so no response was ever sent. The navbar typeahead hits

`Trending`, so any error meant a hung request per keystroke.

Fix: wrap in `asyncHandler`.

🔴 8. Mass assignment in `updateVideo`

```
Object.assign(video, req.body);
await video.save();
```

A client could send `{"views": 999999}`, `{"owner": "<another user>"}` or `{"_id": ...}`. *Fix:* an explicit `UPDATABLE_FIELDS` whitelist.

Lesson: never spread a request body onto a model. Whitelist, never blacklist.

🔴 9. Unescaped user input in `$regex`

```
{ title: { $regex: search, $options: "i" } }
```

`(((((` throws; `.*` or `(a+)+$` causes a full scan or catastrophic backtracking (ReDoS). *Fix:* `escapeRegex()`.

🔴 10. Upload filenames were attacker-controlled and collided

```
filename: (req, file, cb) => cb(null, file.originalname)
destination: (req, file, cb) => cb(null, "./public")
```

Two simultaneous `video.mp4` uploads overwrote each other, `../` in a name could escape the directory, and `./public` was being served by `express.static`. *Fix:* random names, a non-served `tmp/uploads`, a MIME allow-list and a size cap.

🔴 11. Login leaked which accounts exist

"User Not Found, you may register" vs "Invalid Password" lets anyone enumerate users. *Fix:* identical 401 + "Invalid credentials" for both.

🔴 12. `{ email: undefined }` in the login query

`findOne({ $or: [{ email: undefined }, { username }] })` — MongoDB reads `{field: undefined}` as "field does not exist", which can match an unintended

document. *Fix:* build `$or` from present values only.

🟡 13. `apiError` destroyed every stack trace

```
constructor(..., stack = " ") { if (stack) this.stack = stack; }
```

`" "` is truthy. **Every error in the app** had its stack replaced with a space. *Fix:* default to `undefined`.

🟡 14. `getVideoById` dereferenced before the null check

```
video.durationFormatted = formatDuration(video.duration);
if (!video) return res.status(404)...      // ← too late
```

Any missing id threw "Cannot set property of null" → 500 instead of 404. (It was also a no-op: assigning an arbitrary property to a Mongoose document without `toObject()` does nothing.)

🟡 15. `DBNAME` was silently ignored

`${MONGODB_URL}/${DBNAME}` appended the name after the query string. The driver used its default database, `test`. *Fix:* `{ dbName }` as an option, keeping `test` so no data was orphaned.

🟡 16. SocketConn wrote to a different database

It connected with no `dbName`, so presence updates could land in a different database from the one the API reads. *Fix:* same `dbName` on both.

🟡 17. Duplicate subscriptions

No unique index, and a check-then-insert race. Two fast clicks created two documents and the subscriber count was permanently wrong. *Fix:* unique compound index + handle error `11000`.

🟡 18. `searchElastic` referenced an unimported variable

Guaranteed `ReferenceError`. *Fix:* import it, and fall back to Mongo when

Elasticsearch is absent.

🟡 19. `validatePagination` produced a string status code

```
next(new apiError(`${error.status}`, ...));   // "undefined"
```

Status codes must be numbers; this fell through to 500 with a nonsense message. The middleware was also never actually mounted on any route.

🔵 20. Redis could take the whole API down

`await connectRedis()` at boot threw on any Redis problem. *Fix:* never throws, in-memory fallback, `/health` reports the mode.

🔵 21. The cache blocked responses and cached errors

`res.json` was replaced with an `async` function that awaited Redis **before** sending, and it cached 4xx/5xx bodies for the full TTL. *Fix:* respond first, cache after, success only.

🔵 22. `Trending` returned the entire collection

The comment said "pagination, sort by views"; the code was `Video.find(filter)` with no `.limit()`. The navbar typeahead called it on every search.

🔵 23. No `views` increment and no watch history

Both fields existed in the schema and were never written. Every video showed "0 views" forever.

🔵 24. Unstable sort + skip/limit

An unrecognised `?sort=` gave `{}` — natural order, which MongoDB does not guarantee between queries. Rows could duplicate or disappear across pages.

🔵 25. Cloudinary assets were never deleted

Deleting a video removed the document but left the video and thumbnail in Cloudinary forever. Same for replaced avatars (the code literally said

`//todo delete old image`).

🔵 26. `SIGTERM` was not handled

Only `SIGINT`, and it called `process.exit()` immediately. Docker, Kubernetes and Render all send `SIGTERM` — so every deploy killed in-flight requests.

Frontend

🔴 27. `searchUsers` was called but never defined

`pages/searchIndv.jsx` called `searchUsers(query)` with no import and no such function anywhere. `ReferenceError` on the first keystroke. *Fix:* added `GET /users/search` and wired it up.

🔴 28. `videos.map is not a function` blanked the watch page

`/videos/any` returned a paginated envelope while the page expected a bare array. With no error boundary, this unmounted the **entire** React tree → blank white screen. *Fix:* accept both shapes, always hand `setVideos` an array, and add an `ErrorBoundary`.

🔴 29. Every page except `/` had no navigation

Only `/` was inside `MainLayout`. Open a video and there was no way back except the browser button. *Fix:* restructured the route tree — app pages under `MainLayout`, only `/login`, `/register` and the full-screen chat bare.

🟠 30. Passwords rendered in plain text

`type='text'` on the password field in both login and register. Visible to anyone looking at the screen, and no password manager would offer to save it.

🟠 31. Protected pages rendered for signed-out users

Every request inside them 401'd, leaving a blank screen and a console full of errors — or a hard `window.location` redirect that threw away app state. *Fix:* a `<Protected>` gate that toasts and shows a "Log in" panel.

🔵 32. 1.2 MB JS bundle from two never-rendered imports

```
import Aurora from "../assets/aurora.jsx";         // → ogl
import { GridScan } from "../assets/gridscan.jsx"; // → three, face-api, postprocessing
```

A static `import` bundles the module whether or not you use the binding.

Roughly 600 kB downloaded by every visitor for components that were never on screen.

330 kB gzip → 78 kB gzip, combined with route-level `React.lazy`.

🔵 33. Search fired a request per keystroke

No debounce → 429 after about ten characters. *Fix:* 350 ms debounce + `placeholderData` so the dropdown does not flash empty.

🔵 34. Infinite scroll watched the wrong element

`ref={loadMoreRef}` on every card; a ref holds one value, and reassigning `.current` does not re-run the observer effect. *Fix:* one sentinel below the grid.

🔵 35. Thumbnail pop-in while scrolling

Three causes: `fetchPriority="low"` deprioritised every thumbnail; the `<img>` was unmounted and remounted by the hover preview, forcing a re-decode; and there was no placeholder. *Fix:* drop `fetchPriority`, keep the image mounted and layer the video over it, fade in over a shimmer.

🔵 36. Cached images stayed invisible

`onLoad` can fire **before** React attaches the handler for a cached image, so the element stayed at `opacity-0` forever. *Fix:* a `ref` callback checking `complete && naturalWidth > 0`, plus `onError`.

🔵 37. Layout shift in the feed grid

Grid rows size to their tallest item, so a short title made its card shorter and the row shifted as data arrived. *Fix:* fixed-height meta block with the

views/age row pinned, and a skeleton of identical height.

🔵 38. A "Please log in" modal every 5 minutes

A `setInterval` nagged anonymous visitors forever. The feed is public. *Fix:* prompt on an actual 401, and only if there *was* a session.

🔵 39. ~90 redundant requests per page for anonymous visitors

Every page calls `/currentuser` → 401 → refresh → 401 → repeat on every navigation. *Fix:* a 30-second circuit breaker, cleared on login.

🔵 40. Stale `currentUser` cache after login

`staleTime: 60s` meant the cached *signed-out* result was still served after a successful login, so the navbar and `Protected` still thought nobody was logged in. *Fix:* `invalidateQueries(["currentUser"])` before navigating.

🔵 41. Persisted theme pointed at a 404

The chat wallpaper was persisted as an **asset URL**, and Vite fingerprints asset URLs per build. *Fix:* validate against the themes that currently exist and fall back.

🔵 42. `JSON.parse` in a `useState` initialiser

`JSON.parse("undefined")` throws, and a throw in a state initialiser blanks the whole app. `localStorage` itself also throws in private mode. *Fix:* try/catch both.

🔵 43. Menu actions were `console.log` stubs

"Watch later", "Share", "Not interested", "Report" all logged and did nothing. *Fix:* real `watchLater` / `notInterested` persistence, a Saved page, native share with a clipboard fallback, and honest messaging for playlists (which need a schema that does not exist yet).

🔵 44. Unreadable text in dark mode

Tailwind's preflight sets `color: inherit` on form controls, so adding `body { color: var(--text) }` made the login inputs white-on-white. The create-video page hardcoded `text-neutral-800` on a container that sits on black, and its buttons had no text colour at all. *Fix:* explicit colours on always-light cards, theme-aware classes everywhere else.

🔵 45. `useEffectEvent` imported from stable React

Experimental, not exported — silently `undefined` because React is CJS and Vite's interop does not throw on a missing named export.

🔵 46. Dead files and duplicated CSS

Four empty `.jsx` files; two `input[type="file"]` blocks where the second silently overrode the first; the login background is a **4.1 MB JPEG** (still outstanding — see part 9).

The pattern

Most of these fall into five buckets:

1. **Evaluation order** — ESM hoisting, route order, null check after use.
2. **Wrong API shape** — `req.file` vs `req.files`, body vs params, object vs array, array vs envelope.
1. **Missing validation** — regex, mass assignment, filenames, sort values.
2. **Error paths never exercised** — the refresh endpoint, every `catch` that threw from an unwrapped async handler.
1. **Fire-and-forget without a fallback** — Redis, Cloudinary cleanup, caches that were never invalidated.

Bucket 4 is the one worth internalising: **the happy path was tested by hand and the error paths never were.** A single integration test hitting `/refreshtoken` would have caught the worst bug in the repository.

Next: **09 — Design decisions**

9 — Design Decisions

Every non-obvious choice, with the alternative that was rejected and why.
This is the part interviewers probe hardest: anyone can write code, fewer can explain the trade-off they chose.

9.1 Backend

Keep `asyncHandler` even though Express 5 handles promises

Express 5 auto-forwards rejected promises, so the wrapper is technically redundant. Kept because it is explicit, costs nothing, and the codebase mixes wrapped and unwrapped handlers — a single convention is worth more than the six lines saved.

Redis fails open, authorisation fails closed

Component	On failure	Why
Redis cache	serve uncached	it is an optimisation
Rate limiter	let the request through	locking everyone out is worse than brief over-permissiveness
Elasticsearch	fall back to Mongo regex	degraded search beats no search
`verifyJWT`	**reject**	an auth check that fails open is not a check

The rule: **fail open for availability concerns, fail closed for security concerns.** Being able to state that distinction is the point.

An in-memory fallback rather than just skipping the cache

A `Map` with TTLs is ~80 lines. The alternative — `if (!redis) return next()` — means a Redis outage removes rate limiting entirely, which is exactly when you are most likely to be under load. The fallback is per-instance rather than shared, which is a real limitation, and `/health` surfaces which mode you are in so it is never silent.

Preserve `test` as the database name

The `DBNAME` bug meant all real data lives in the default `test` database. "Fixing" it properly would have pointed the app at an empty database and

appeared to delete everything. The fix makes the name **explicit and correct** while keeping the same target.

Principle: a correctness fix that moves live data is a migration, not a fix.

`POST /videos/any` kept as an alias

```
export const any = getVideosByOwner;
```

The old name is meaningless, but the frontend calls it. Aliasing lets the implementation improve without a coordinated deploy. (The response shape did change, which broke the watch page — a reminder that renaming is safe and reshaping is not. The consumer now accepts both.)

Watch history on the user, messages in their own collection

Both are "a list of things". The difference is **bounded vs unbounded**.

Watch history is capped at 100 with `$slice`, always read with the user, and never queried in reverse — so embedding ids is right. Messages grow without limit and would eventually hit the 16 MB document ceiling.

`conversationId` derived, not stored

`[a, b].sort().join("_")` is symmetric, needs no extra collection, and cannot drift out of sync. The cost: it does not generalise to group chats, which would need a real `Conversation` document. A deliberate trade for a 1-to-1 messaging product.

Rate limit by user when known, IP otherwise

IP-only punishes everyone behind a shared NAT (a school, an office, a mobile carrier). User-only is unusable for login, which is pre-authentication.

`req.user?._id ?? req.ip` covers both — and `trust proxy` makes the IP real.

Tighter limits on credential endpoints

30/20s for reads, 10/60s for login and register. Those are the brute-force surface, and they should be slow; a feed endpoint should not be.

9.2 Frontend

Route structure: two layouts, not one

Layout	Pages	Why
`MainLayout`	feed, channel, watch, saved, search, settings, inbox	needs the navbar
`AuthLayout`	login, register, **chat view**	full-bleed, no chrome

The chat view is the interesting one. It renders its *own* `fixed top-0` header and `fixed bottom-0` composer, so a navbar above it would double up. The inbox (`/message`) is a normal list and keeps the navbar.

`<Protected>` renders a panel instead of redirecting

A hard redirect loses the URL the user wanted, and makes "log in and come back" impossible. Rendering in place preserves the route, passes `state.from` to the login page, and the URL stays shareable.

It also distinguishes **three** states, not two:

```
loading  → spinner         (NOT the signed-out panel)
no user  → toast + panel
user     → the page
```

Collapsing loading into "signed out" makes the panel flash on every refresh before the session check returns — a small thing that makes an app feel broken.

Toasts over a DOM `CustomEvent`, not React context

```
window.dispatchEvent(new CustomEvent("app-toast", { detail: { ... } }));
```

Because the axios interceptor needs to raise toasts and it lives **outside** the component tree. A context-based API would force every non-React module to receive a callback. The interceptor already dispatched a `rate-limit` event that nothing listened for — this gave it a home.

The feed, channels and watch pages stay public

Only personal pages are gated. A social platform whose content requires login has no discovery path and no shareable links. `optionalJWT` lets the watch page record history when it knows who you are, without requiring it.

Optimistic hide for "Not interested", server round trip for "Watch later"

"Not interested" is low-stakes and benefits from feeling instant, so the card disappears immediately and rolls back on error. "Watch later" shows a toast on confirmation because the user expects to find it in a list afterwards — claiming success before the server agrees would be a lie if it failed.

"Save to playlist" is disabled, not fake

Playlists need a collection with name, ordering and privacy. That schema does not exist. A button that silently does nothing is worse than a disabled one, and a disabled one is more honest than a fake success toast.

Fixed-height meta block, pinned views/age row

CSS grid rows size to their tallest item. Without a reservation, a one-line title made its card shorter and the whole row shifted as thumbnails and titles streamed in. The skeleton uses the identical `h-[102px]`, so swapping placeholder for data causes **zero** layout shift.
The title itself is *not* fixed-height — the channel name sits naturally underneath it — and only the views/age row is absolutely positioned at the bottom. That keeps the natural reading rhythm while giving the eye a stable line to scan across cards.

Skeletons over spinners

A spinner says "something is happening". A skeleton says "content of *this shape* is arriving", and if it matches the real layout it eliminates layout shift. Used for the feed, search results and the Saved list; the spinner is kept only for "fetching the next page", where the layout is already stable.

CSS variables alongside the `darkMode` prop

The components thread a `darkMode` boolean through props. Rewriting all of them to Tailwind's `dark:` variant would have been a large, risky change. Instead `:root` / `body.dark` tokens give plain CSS one source of truth while the prop-driven classes stay untouched — the two systems coexist and `document.body.className` already drove the switch.

`.input` deliberately ignores dark mode

It is used only on the register form, which always renders on a white card

over a photo. Making it follow the theme produced white text on white. **A component that always renders on a fixed background should not follow a global theme.**

`prefers-reduced-motion` honoured globally

The app leans on transitions, hover-to-play previews and particle backgrounds. For people with vestibular disorders that is actively unpleasant, and it is three lines of CSS.

Visible focus rings restored

Tailwind's preflight removes the UA outline and none of the custom buttons added one back, so keyboard navigation was invisible. A `:focus-visible` rule restores it without showing a ring on mouse clicks.

9.3 Performance

Dead imports were the whole problem

Route-level `React.lazy` is the textbook answer and it helped — but it took the bundle from 1,207 kB to 910 kB. Deleting two never-rendered imports took it to **273 kB**.

The lesson is to **measure before optimising**. The obvious fix was worth 25%; the unglamorous one was worth 70%.

`loading="lazy"` kept, `fetchPriority="low"` removed

`loading="lazy"` is correct: it defers off-screen images and the browser starts them ~1250 px ahead. `fetchPriority="low"` layered an *additional* deprioritisation on top, which is what made thumbnails arrive visibly late while scrolling.

The view counter is fire-and-forget

```
Video.updateOne({ _id }, { $inc: { views: 1 } }).catch(() => {});
```

Awaiting it would add a database round trip to every page load for a number nobody checks for exactness. At scale this should be batched in Redis and flushed periodically — the `.catch()` is mandatory either way, since an unhandled rejection crashes Node.

9.4 Known limitations (say these before you are asked)

1. **No tests.** The single highest-value next step. One integration test on `/refreshtoken` would have caught the worst bug in the repo.
1. **Socket.IO is unauthenticated.** Anyone can emit `register-user` with any id. Needs a JWT handshake middleware.
1. **Prop drilling in `App.jsx`.** Eight pieces of state threaded through three levels. Needs split contexts.
1. `isLoggedIn` **duplicates the query cache.** Derived state that can drift.
2. ** `KEYS videos:*` for invalidation.** O(n) and blocking. Needs a generation counter.
1. `skip` / `limit` **pagination.** O(offset), and unstable under inserts.
2. **Uploads proxy through the API.** A 200 MB video occupies a server for the whole transfer. Signed direct-to-Cloudinary uploads are the single biggest available win.
1. **The login background is a 4.1 MB JPEG.** Should be a resized WebP/AVIF with a `<picture>` fallback — probably under 150 kB.
1. **No CSRF tokens.** Mitigated by `sameSite`, but defence in depth would add them.
1. **No pagination on the inbox.** `getConversations` aggregates every message the user has ever sent or received on each load.
1. **Presence is in-process.** Breaks across multiple instances without the Socket.IO Redis adapter.
1. **No structured logging.** `console.log` is not queryable; pino/winston with request ids would be.

Being able to list these unprompted is worth more than having fixed half of them silently. It demonstrates you know where the bodies are.

Next: **10 — Interview Q&A**

10 — Interview Q&A

Answers phrased the way you would actually say them, each anchored to this project.

Project walkthrough

"Walk me through your project."

> It's a video-sharing platform with real-time messaging — three services sharing a MongoDB Atlas cluster. A React SPA on Vite, an Express REST API for auth, videos and message persistence, and a separate Socket.IO service for presence and live delivery.
> The split I'd call out: messages are **written** through the REST API so they're validated, rate-limited and durably stored, and **delivered** through the socket service. The socket layer is a transport, not a source of truth — if it dies, nothing is lost, the next history fetch still returns everything.
> Auth is JWT in httpOnly cookies with refresh-token rotation. Media goes to Cloudinary. Redis caches feed pages and backs the rate limiter, but it's deliberately optional — if it's unreachable the API falls back to an in-process cache and keeps serving.

"What was the hardest bug?"

> Redis kept connecting to localhost even though `REDIS_URL` pointed at Upstash. The value was right in `.env` and right in the shell.
> The cause was ES module evaluation order. `index.js` imported `redis.js` at the top and called `dotenv.config()` in the body — but imports are hoisted and the whole dependency graph is evaluated *before* the importing module's body runs. So `redis.js` read `process.env.REDIS_URL` while it was still undefined, and `createClient` defaults to localhost.
> What made it hard is that it was silent. No error about missing config — just a connection refused to an address nobody configured. The same bug was also breaking CORS and Cloudinary, which looked like three unrelated problems.

> The fix was moving `dotenv.config()` into its own module imported first, since module bodies evaluate in source order. The broader lesson is that `import` is a hoisted declaration, not an in-place statement — which is exactly the difference from `require()`.

"What would you do differently?"

> Write tests first. There were no tests at all, and the worst bug — `refreshAccessToken` referencing two undefined variables, so every session silently died when the access token expired — would have been caught by a single integration test, or honestly just by running ESLint over the backend.
> Architecturally, I'd make uploads go directly to Cloudinary with a signed URL instead of proxying a 200 MB video through the API server. And I'd use cursor pagination from the start rather than `skip` / `limit`.

JavaScript

"`var` vs `let` vs `const`?"

> `var` is function-scoped and hoisted initialised to `undefined`. `let` and `const` are block-scoped and hoisted but in the temporal dead zone, so reading before the declaration throws. `const` prevents reassignment of the binding, not mutation of the value — `const arr = []; arr.push(1)` is fine.
> The practical consequence is closures in loops: `var` gives one shared binding so three `setTimeout`s all log 3; `let` is re-bound per iteration so they log 0, 1, 2.

"Explain the event loop."

> JavaScript runs on one thread with a queue. The key rule is that the entire **microtask** queue drains between every **macrotask**. Promise callbacks and `queueMicrotask` are microtasks; timers, I/O and `setImmediate` are macrotasks. `process.nextTick` jumps ahead of promises.
> So `setTimeout(fn, 0)` runs *after* an already-resolved promise's `.then`.
> Practically, in this project it means a long synchronous operation — an unbounded `Video.find()` with no limit, say — blocks the entire server, not

> just that request. That's why the `Trending` endpoint returning the whole collection was an availability risk, not just a slow query.

"`==` vs `===`?"

> `===` compares type and value. `==` coerces first, with rules that aren't worth memorising: `"" == 0` is true, `null == undefined` is true but `null == 0` is false, `NaN != NaN`.
> Always `===`. The one idiomatic exception is `x == null` as a shorthand for "null or undefined" — though `??` usually expresses that better now.

"What's a closure?"

> A function plus the lexical environment it was created in. The inner function keeps access to the outer scope's variables after the outer function has returned.
> In this project every middleware is one. `rateLimiter({ limit: 10 })` returns a handler that still sees `limit` — that's how `rateLimit` and `authRateLimit` are two functions with different captured configuration from the same factory.

"`null` vs `undefined`?"

> `undefined` means "never assigned" — the default for uninitialised variables, missing parameters, and absent object properties. `null` is an explicit "assigned, and the value is nothing".
> It matters in MongoDB: `{ field: undefined }` is treated as "field doesn't exist" and can match unintended documents, which was a real bug in the login query here. `{ field: null }` matches documents where it's actually null.

Node & Express

"What is middleware?"

> A function with `(req, res, next)` that sits in a chain. It can read and mutate the request, end the response, or call `next()` to continue. Error middleware takes **four** parameters — Express identifies it by arity, so dropping the unused `next` silently turns it into ordinary middleware and

> your error handler never runs.
> Order is semantics: CORS first, because an error thrown before it produces a response with no CORS headers that the browser reports as a generic network failure. Body parsers before routes, or `req.body` is undefined. Error handler last.

"How do you handle async errors in Express?"

> In Express 4 you can't just throw from an `async` handler — Express only catches synchronous throws. An async function returns a rejected promise, Express ignores the return value, so nothing responds and the request hangs until the client times out.
> The fix is a wrapper: `Promise.resolve(handler(req,res,next)).catch(next)`.
> Express 5, which this project uses, forwards rejections automatically — so the wrapper is now belt-and-braces. I kept it because the codebase mixes styles and one convention is clearer.
> This was a live bug here: two route handlers threw inside a `catch` block without the wrapper, and the navbar's typeahead hit one of them, so any error hung a request per keystroke.

"What is CORS?"

> A browser mechanism that restricts cross-origin reads. The server declares who may read its responses via `Access-Control-Allow-Origin`.
> Two rules that catch people out. First, `*` is forbidden with `Access-Control-Allow-Credentials: true` — so with cookies you must echo the origin back from an allow-list. Second, non-simple requests trigger a preflight `OPTIONS`, and if preflight fails the real request is never sent.
> And it's enforced by the browser, not the server — it protects *users* from other *sites*, it's not a defence against scripted clients.

"Streams?"

> Four kinds: readable, writable, duplex, transform. The point is processing data in chunks instead of loading it all into memory.
> Directly relevant here: multer's disk storage streams uploads to disk, so a 200 MB video uses constant memory. Memory storage would hold the whole thing in RAM — fine for an avatar, fatal for video.

MongoDB

"Embed or reference?"

> Embed when the data is read together, bounded in size, and doesn't need independent queries. Reference when it's unbounded, shared, or queried on its own.
> In this project: watch history is an embedded array of ids capped at 100 with `$slice` — bounded, always read with the user, never queried in reverse. Messages are a separate collection because they're unbounded and would eventually hit the 16 MB document limit. Subscriptions are their own collection because it's many-to-many and queried from both directions.
> The 16 MB limit is the hard constraint behind all of it.

"How do indexes work? What's the ESR rule?"

> A B-tree on the indexed fields, so a lookup is O(log n) instead of a collection scan.
> For compound indexes the field order is Equality, Sort, Range. The feed query matches `{isPublished: true}` and sorts by `createdAt` descending, so the index is `{isPublished: 1, createdAt: -1}`. Get it wrong and Mongo has to sort in memory — and an in-memory sort over 32 MB fails outright.
> A compound index also serves any prefix of itself, so `{a,b,c}` covers `{a}` and `{a,b}` but not `{b}`.

"`save()` vs `findByIdAndUpdate()`?"

> `save()` runs document middleware; update operators talk to the database directly and skip it entirely.
> That's load-bearing here. The password is hashed in a `pre("save")` hook, so changing a password with `findByIdAndUpdate` would store it in **plaintext**.
> The change-password controller deliberately assigns and calls `.save()`.
> Related subtlety: the hook guards on `isModified("password")`. Without it, the `save()` that stores a new refresh token would re-hash the already-hashed password and permanently lock the account out.

"How would you fix slow pagination?"

> `skip(n)` walks and discards n documents, so it's O(offset) — at page 1000 with limit 20 the database reads 20,020 documents to return 20. It's also unstable: an insert between page 1 and page 2 shifts everything and the user sees duplicates.
> Cursor pagination fixes both — instead of `?page=3` you pass the last item you saw and query `{createdAt: {$lt: cursor}}`. O(log n) via the index, and stable under inserts. You lose "jump to page 7", which is exactly the right trade for an infinite feed.

React

"Why do keys matter?"

> React matches children between renders by key. With index keys, deleting the first item renumbers everything — React thinks item 0 *changed* rather than *was removed*, so it reuses the DOM node and its component state. A typed input value or a playing video attaches to the wrong row.
> Index keys are only safe for an append-only list that's never reordered or filtered.

"`useEffect` — when does cleanup run?"

> Before the next effect run, and on unmount.
> That's what makes debouncing work: the effect sets a timer, cleanup clears it, so each keystroke cancels the pending one and only the last survives. Without it the navbar fired a search per keystroke and tripped the server's rate limiter after about ten characters.

"What's a stale closure?"

> An effect or callback captures variables from the render it was created in. With an empty dependency array it captures the *first* render's values forever — so an interval logging `count` always logs 0.
> Fixes: list the dependency, or use the functional updater `setCount(c => c + 1)` so you never read the current value at all.

"`useMemo` vs `useCallback` vs `useRef`?"

> `useMemo` caches a computed value, `useCallback` caches a function identity, `useRef` is a mutable box that persists across renders *without* triggering one.
> `useRef` has two distinct jobs here: holding a DOM node for the IntersectionObserver, and holding a `toasted` boolean in `<Protected>` — if that were state, setting it would re-render, which re-runs the effect, which toasts again.

"How would you improve performance?"

> I'd measure first. Here the build was 1.2 MB and the obvious answer — route-level code splitting with `React.lazy` — only got it to 910 kB.
> The real win was two `import` statements for components that were never rendered. A static import bundles and executes the module whether or not you use the binding, so about 600 kB of three.js, face-api and WebGL libraries shipped to every visitor for nothing. Deleting them plus the code splitting took it to 273 kB — 330 kB gzip down to 78.
> Beyond bundle size: `IntersectionObserver` instead of scroll listeners, debounced search, TanStack Query so three components share one request, and fixed-height skeletons so there's no layout shift.

"Error boundaries?"

> A class component with `getDerivedStateFromError` and `componentDidCatch` — there's no hook equivalent. It catches render-phase errors in its subtree, but **not** event handlers, async code, or SSR.
> I added one after a type error on the watch page — `videos.map is not a function` — unmounted the entire React tree and left a blank white screen. Now the failure is contained to that route with a retry button.

Auth & security

"Why bcrypt and not SHA-256?"

> SHA-256 is designed to be fast — billions of guesses per second on a GPU, which is exactly what you don't want. bcrypt is deliberately slow and tunable: cost 10 means 2^{10} iterations, roughly 100 ms, and each increment

> doubles it. It also generates a per-user salt automatically and embeds it in the hash, so identical passwords produce different hashes and rainbow tables are useless.
> Argon2id is the current recommendation because it's memory-hard too, so it also resists GPU and ASIC attacks.

"Why two tokens?"

> A JWT can't be revoked — verification is pure signature maths with no database lookup, which is what makes it scale and also what makes it a problem.
> So you make the unrevocable token short-lived, and gate renewal behind a long-lived refresh token that **is** checked against the database. Logout unsets the stored refresh token, so it dies immediately; the access token survives at most one more expiry window.
> I also rotate: every refresh issues a new refresh token and overwrites the stored one, so a stolen token is single-use.

"Cookies or localStorage for tokens?"

> httpOnly cookies. `localStorage` is readable by any JavaScript on the page, so one XSS takes every token. An httpOnly cookie can't be read by script at all.
> The trade is CSRF, since cookies are sent automatically — but `sameSite` is a one-line mitigation, and there's no equivalent one-liner for "a script can read your token". XSS is also more common than CSRF in practice.
> `sameSite` needs care: production has the API and SPA on different sites so it must be `none`, which requires `secure: true`. Locally both are `localhost`, so `lax` works over plain HTTP. Getting that wrong is the classic "works locally, broken in production" auth bug.

"How do you prevent injection?"

> Never concatenate user input into something you then execute.
> MongoDB isn't immune just because it isn't SQL. User input going straight into `$regex` was a real bug here — `(((((` throws, and `(a+)+$` causes catastrophic backtracking. I escape the metacharacters.
> There's also query-object injection: `{email: undefined}` is read as "field

> doesn't exist" and can match the wrong document, so I build the `$or` from present values only. And mass assignment — `Object.assign(video, req.body)` let a client set `views` or `owner`, so that's a whitelist now.

"Rate limiting — how, and what's wrong with it?"

> Redis `INCR` with a TTL: increment a key per user or IP, set the expiry on the first hit, reject over the limit.
> The flaw is the boundary burst — a fixed window allows 2× the limit across the boundary, 30 requests at 19.9 s and another 30 at 20.1 s. A sliding window or token bucket fixes that; token bucket is the usual production choice because it also allows deliberate bursts.
> I made it fail **open**: if Redis is down, requests go through. Locking every user out of the product because a cache is unavailable is worse than brief over-permissiveness. For an authorisation check I'd do the opposite.

System design

"How would you scale this to a million users?"

> I'd go in order of what actually hurts.
> First, uploads. A 200 MB video currently streams through the API server to Cloudinary, tying up a process for the whole transfer. Signed direct uploads move those bytes browser-to-Cloudinary and the API only stores the resulting URL. Single biggest win.
> Second, pagination — `skip` / `limit` is O(offset). Cursor-based for the feed.
> Third, make the API genuinely stateless so I can run N instances behind a load balancer. That means the Socket.IO Redis adapter for presence, since `onlineUsers` is currently an in-process `Map` that breaks the moment there are two instances.
> Then caching: cache the user lookup that `verifyJWT` does on every request, and replace `KEYS videos:*` invalidation with a generation counter, because `KEYS` is O(n) and blocks Redis.
> Reads dominate a feed, so MongoDB read replicas, then sharding on `owner` for videos if a single replica set isn't enough. Video delivery moves to a CDN with HLS adaptive bitrate.

> At the far end the feed itself becomes the problem and you'd precompute timelines with fan-out-on-write rather than querying per request.

"How does your real-time messaging work?"

> Socket.IO with rooms. On connect the client emits `register-user` and the socket joins a room named after the user id, so delivering a message is `io.to(receiverId).emit(...)` — no socket-id bookkeeping and automatic multi-device fan-out.
> Presence uses a `Map<userId, Set<socketId>>` rather than a single socket id, because one user has multiple tabs. The online/offline transition only fires when the set becomes empty — it's reference counting, and it's the thing most presence implementations get wrong.
> Messages are persisted through the REST API, not the socket, so validation, rate limiting and durability live in one place.
> I'd call out two gaps honestly: the socket isn't authenticated, so anyone can claim any user id — that needs a JWT handshake middleware. And presence is in-process, so it needs the Redis adapter before running more than one instance.

"Why cache, and how do you invalidate?"

> Feed pages are read constantly and change rarely, so cache-aside with a 5-minute TTL on `videos:<page>:<limit>:<sort>`, invalidated on create, update and delete.
> Two things I'd flag. The cache key includes the user id — without that, a personalised response cached for one user gets served to another, which is a data-leak class of bug.
> And my invalidation uses `KEYS videos:*`, which is O(n) over the whole keyspace and blocks Redis since it's single-threaded. Fine at this scale, wrong at any real scale — I'd use a generation counter so bumping one integer orphans every old key and they expire naturally.

"What happens if Redis goes down?"

> Nothing user-visible. It's a cache and a rate-limit counter, not a source of truth, so I wrote the client to never throw — every call falls back to an in-process `Map` with TTLs.

> The degradation is real and worth naming: the fallback is per-instance, so hit rate drops and rate limits become per-instance rather than global. `/health` reports which mode you're in, so it's never a silent surprise. The original code did `await connectRedis()` at boot and would crash the entire API if Redis was unreachable — a cache outage became a total outage.

Closing questions to ask them

- How do you handle database migrations and schema changes?
- What does your testing pyramid actually look like in practice?
- How is on-call structured, and what's a typical incident?
- What's the deploy process — how long from merge to production?
- Where does the team feel the most technical debt?